



南京大學

研究生畢業論文
(申請碩士學位)

論文題目 面向大規模動態圖的

圖神經網絡優化機制研究

作者姓名

專業名稱

研究方向

指導教師

2023 年 4 月 8 日

学 号：

论文答辩日期：XXXX 年 XX 月 XX 日

指 导 教 师： (签字)

Research on Graph Neural Network Optimization Mechanism for Large-Scale Dynamic Graphs

by

Supervised by

A dissertation submitted to
the graduate school of Nanjing University
in partial fulfilment of the requirements for the degree of
MASTER
in



Department of Computer Science and Technology
Nanjing University

April 8, 2023

南京大学研究生毕业论文中文摘要首页用纸

毕业论文题目： 面向大规模动态图的图神经网络优化机制研究

专业 2020 级硕士生姓名：

指导教师（姓名、职称）：

摘 要

由于现实世界中的大多数数据都可以用图表示，因此近年来针对图表示学习的研究越来越受到关注。通过将高维图数据转化成低维的表征向量，图表示学习可以有效地对图的信息进行存储并快捷地访问图中实体的关键知识。利用学习到的图表征数据可以为社交网络行为分析、节点分类、链路预测和聚类等下游任务提供帮助。随着深度学习的快速发展，研究人员提出了基于图神经网络的图表示学习方法，其使图中的节点能够同时学习图的结构信息以及邻域节点的特征信息，从而得到高质量的低维表征。针对图神经网络上的图表示学习，本文主要关注两个问题：大规模时序图场景下的图表示学习方法和分布式场景下的大规模图的图神经网络优化训练方法。

真实世界中的图数据规模往往十分庞大且处于动态变化的状态，而现有的图神经网络要么无法处理动态时序图上的图表征学习，要么面对大规模图数据集时训练效率低下。为了解决大规模动态图上的图表征学习问题，本文提出了一种基于图摘要的大规模时序图表示学习方法——GSAERU 模型。GSAERU 模型由“图表征学习模块”和“时序学习模块”两部分构成。首先将动态时序图建模为各时刻上原图快照的序列，接着在单时间步上将图快照输入图表征学习模块。该模块通过图摘要技术对原图进行压缩，并将压缩后的新图输入图自编码器，利用和原图的重构误差训练原图节点的表征。在获得单时间步上的图表征后将其输入时序学习模块，并通过一个循环神经网络学习原图在时间维度上的特征信息与依赖关系，从而得到大规模时序图的高质量表征。基于在四个真实世界的大规模图数据集上的大量实验表明，GSAERU 模型能够高效生成时序图的表征。与传统的图神经网络算法相比，其平均内存消耗和平均训练时长只有其 9.86% 与 13.27%，且与性能最好的算法在大部分指标下的差距都不超过 3.5%。

使用分布式技术加速神经网络训练也是如今热门的研究课题，因为在处理大规模数据时，单个设备（如 CPU，GPU）有限的内存和计算资源往往会成为训练瓶颈。而分布式技术可以提供更多的计算资源来提高训练效率。然而由于图数据的不规则性，传统分布式机器学习方法中的子任务划分和模型训练方法难以直接应用于图神经网络。针对大规模图流数据场景下的分布式图神经网络训练优化问题，本文提出了基于图摘要的分布式图神经网络优化训练算法——DSGNN 模型。DSGNN 模型采用“领导——工作者”工作模式，首先由领导节点利用一个基于边分割的图流划分算法对原图进行划分并将划分子图分配给各个工作节点。工作节点随后执行一个基于图摘要技术的小批量训练并将训练结果通过一个注意力层返回给领导节点，最后由领导节点汇总结果并计算梯度，并根据梯度同步更新所有计算节点上的模型。基于四个真实世界的大规模图数据集上的大量实验，结果表明 DSGNN 模型可以大大加速 GNN 模型的训练，并且在链路预测任务上的 F1 分数显著高于其它分布式图神经网络模型。

关键词： 图表示学习；深度学习；图神经网络；分布式技术

南京大学研究生毕业论文英文摘要首页用纸

THESIS: Research on Graph Neural Network Optimization Mechanism
for Large-Scale Dynamic Graphs

SPECIALIZATION: _____

POSTGRADUATE: _____

MENTOR: _____

Abstract

Since most data in the real world can be represented by graphs, research on graph representation learning has attracted increasing attention in recent years. By transforming high-dimensional graph data into low-dimensional vectors, graph representation learning can store graph information and access key information of the graph effectively. Utilizing the learned graph representation data can benefit downstream tasks such as social network, node classification and so on. With the development of deep learning, researchers have proposed a graph representation learning method based on graph neural network, which enables nodes in the graph to simultaneously learn the structural information of the graph and the feature information of neighboring nodes, thereby obtaining high-quality and low-dimensional representations. For graph representation learning on graph neural networks, this paper focuses on two issues: graph representation learning methods in large-scale sequential graphs and graph neural network optimization training methods for large-scale graphs in distributed scenarios.

The scale of graph in the real world is often large and keeps changing, and the existing GNN models are either unable to handle graph representation learning on dynamic sequential graphs, or the training efficiency is low on large-scale graph. To solve the problem of graph representation learning on large-scale dynamic graphs, this paper proposes a large-scale sequential graph representation learning method based on graph summarization—GSAERU model. The GSAERU model consists of two parts: "graph representation learning module" and "time sequence learning module". First, the dynamic sequence graph is modeled as a sequence of snapshots of the original graph at each moment, and then the graph snapshots are input into the graph representation learning module at a single time step. This module compresses the original graph through

the graph summarization technology, and inputs the compressed new graph into the graph autoencoder, and uses the reconstruction error with the original graph to train the representation of the nodes of the original graph. After the graph representation on a single time step is obtained, it is input into the timing learning module, and the feature information and dependencies of the original graph in the time dimension are learned through a recurrent neural network, so as to obtain a high-quality representation of large-scale timing graphs. Extensive experiments on four real-world large-scale graph datasets show that the GSAERU model can efficiently generate representations of timing graphs. Compared with the traditional graph neural network algorithm, its average memory consumption and average training time are only 9.86% and 13.27%, and the gap with the best performing algorithm in most indicators does not exceed 3.5%.

Using distributed technology to accelerate neural network training is also a hot research topic nowadays. The limited memory and computing resources of a single device often become a training bottleneck when processing large-scale data. Distributed technology can provide more computing resources to improve training efficiency. However, due to the irregularity of graph data, the subtask division and model training methods in traditional distributed machine learning methods are difficult to be directly applied to graph neural networks. Aiming at the optimization problem of distributed graph neural network training in large-scale graph streaming data scenarios, this paper proposes a distributed graph neural network optimization training algorithm based on graph summaries——DSGNN model. The DSGNN model adopts the "leader-worker" mode. First, the leader divides the original graph using a graph flow partition algorithm based on edge segmentation and assigns the divided subgraphs to each worker. The workers perform a mini-batch training based on graph summarization technology and return the training results to the leader through an attention layer. Finally, the leader summarizes the results and calculates the gradient, and synchronously updates the models on all computing nodes according to the gradient. Based on extensive experiments on four real-world large-scale graph datasets, the results show that the DSGNN model can greatly accelerate the training of GNN models, and the F1 score on the link prediction task is significantly higher than other distributed graph neural network models.

keywords: graph representation learning, deep learning, graph neural network, distributed technology

目 录

中文摘要	i
英文摘要	iii
目 录	v
插图清单	vii
附表清单	ix
第一章 绪论	1
1.1 研究背景	1
1.2 研究问题	3
1.2.1 大规模时序图表示学习的效率问题	3
1.2.2 分布式场景下的图神经网络训练问题	4
1.3 主要贡献	4
1.3.1 基于图摘要的大规模时序图表示学习算法	4
1.3.2 基于图摘要的分布式图神经网络优化训练算法	5
1.4 论文结构	6
第二章 相关工作	9
2.1 图神经网络	9
2.1.1 基于静态图的方法	9
2.1.2 基于动态图的方法	11
2.1.3 基于分布式的方法	12
2.2 图摘要	13
2.2.1 基于位压缩的方法	14
2.2.2 基于简化的方法	14
2.2.3 基于 sketch 的方法	15
第三章 基于图摘要的大规模时序图表示学习算法	17
3.1 研究动机	17
3.2 问题建模	19
3.3 模型设计和算法描述	20

3.3.1 模型框架	20
3.3.2 图表征学习模块	20
3.3.3 时序学习模块	25
3.4 实验分析	27
3.4.1 实验设置	27
3.4.2 数值结果	29
3.4.3 数据可视化	32
3.4.4 超参分析	33
3.4.5 消融实验	34
3.5 本章小结	36
第四章 基于图摘要的分布式图神经网络优化训练算法	37
4.1 研究动机	37
4.2 问题建模	40
4.3 模型设计和算法描述	40
4.3.1 模型框架	41
4.3.2 基于 Sketch-DBH 算法的子图划分	41
4.3.3 基于图摘要技术的小批量训练	44
4.3.4 全局梯度计算与模型参数更新	48
4.4 实验分析	48
4.4.1 实验设置	48
4.4.2 数值结果	50
4.4.3 数据可视化	53
4.4.4 超参分析	53
4.5 本章小结	55
第五章 总结与展望	57
5.1 工作总结	57
5.2 前景展望	58
参考文献	59
致 谢	69
简历与科研成果	71
版权及论文原创性说明	73
《学位论文出版授权书》	75

插图清单

2-1	Count-Min Sketch 数据结构原理示意图	15
3-1	真实场景中用户与博文互动并相互关注的动态网络.....	18
3-2	现有时序图预测方法原理及瓶颈	19
3-3	静态图与时序图的对比示例	20
3-4	GSAERU 模型框架	21
3-5	SimHash 方法计算 $h(v_i)$ 流程示例	22
3-6	门控循环神经网络	26
3-7	聚类效果展示	33
3-8	超参分析	34
4-1	分布式 GNN 训练通用框架图	38
4-2	分布式 GNN 框架的四种分类	39
4-3	DSGNN 整体框架图	41
4-4	图流数据处理与更新存储结果示例	43
4-5	图摘要示例	46
4-6	基于图 Sketch 上损失函数值的注意力机制原理	47
4-7	DSGNN+GraphSaint 训练过程可视化结果	53
4-8	超参分析	54

附表清单

- 3-1 数据集的统计数据 28
- 3-2 各模型训练时内存占用与训练时间比较..... 30
- 3-3 图表征学习算法在链路预测任务上的性能比较 31
- 3-4 图表征学习算法在节点多分类任务上的性能比较 32
- 3-5 消融实验：不同模型结构对 GSAERU 模型性能的影响分析 35

- 4-1 数据集的统计数据 49
- 4-2 GNN 模型与其分布式版本训练时内存占用与训练时间比较 51
- 4-3 图表征学习算法在链路预测任务上的性能比较 52
- 4-4 不同分布式 GNN 框架运行 GCN 模型的 F1 值（%）对比 52

第一章 绪论

1.1 研究背景

图是一种通过节点集合与边集合来描述数据中的实体对象及其抽象关联关系的数据结构，有着强大的非结构化数据的表示能力。因此图状数据有着相当广泛的应用，如社交网络中用户的关注关系，电子商务平台中用户与商品的推荐关系，金融领域中的账户汇款关系等，这些不仅与实际生活息息相关，也在各自领域发挥着重要的作用。随着大数据时代的到来，大量的数据以非结构化的形式涌现，截至 2022 年，全球社交媒体用户超过 46.2 亿，相当于全球总人口的 58.4%。其中 Facebook 的月活跃用户数达到 29.36 亿，微信月活跃用户达到 12.88 亿，YouTube 的潜在广告覆盖面为 24.76 亿，TikTok 广告每月覆盖 10.2 亿成年人，这些平台每天都在产生着数以万亿计的用户内容与交互联系，高效地处理分析图数据能够帮助我们挖掘其中丰富的有价值信息，为研究与生产提供新的视角和工具。因此在 DB-Engines 统计的数据库流行趋势排名中，相比于关系数据库、文档数据库等常见数据分析工具，图数据库是自 2013 年以来增速最快的分类。依据国际知名调研机构 Gartner 的预测，全球图分析应用将以每年 100% 的速度迅猛增长，到 2025 年使用图技术进行数据分析和创新的比例将由 2021 年的 10% 提升至 80%，包括图数据库管理系统在内的图技术市场将增长到 32 亿美元，复合年增长率为 28.1%。显而易见，对于大规模图数据的处理和分析已经成为下一个技术创新的前沿领域，亟需针对于图数据的有效分析手段。

针对这一问题，科研人员开发了各种图嵌入（Graph Embedding）技术来将原始图数据转换为统一维度的向量，这些向量中保存着原图的拓扑信息，可以用来为社交网络分析、节点分类、链接预测和聚类等下游任务提供重要支撑。这一过程也被称为图表示学习（Graph Representation Learning）。经典的图嵌入方法旨在将高维图数据的维度降低为低维表征，同时保留原图的拓扑信息。它们包括基于线性方法的主成分分析（Principal Component Analysis, PCA）[1] 和基于非线性方法的等距特征映射 [2]，基于随机游走的方法 [3] 和基于矩阵分

解的方法 [4]。虽然这些方法能学习到原图的拓扑信息，但却无法针对原图携带的特征信息进行学习。

近年来随着深度学习的普及，使用神经网络模型进行表示学习成为了研究热门，这是因为其能够从规则的数据结构中提取特征信息。然而非结构化的图数据有着不相互独立的节点，每个节点拥有不同数量的连接关系，传统神经网络中的一些重要操作无法直接应用到图数据，为了解决这一问题，图神经网络应运而生。

图神经网络（Graph Neural Networks, GNNs）是一种基于深度学习处理图数据的方法，它将图上的传播操作和神经网络结合起来，能够学习到图数据中的节点与图结构的内在规律和更加深层次的语义特征，弥补传统深度学习在可推理和可解释性的局限性。在实际应用中，基于图神经网络的图表示学习方法表现明显优于其它传统图表示学习方法 [5]。

尽管 GNN 模型在处理图数据上展现出了优秀的能力，但如何高效地大规模图数据上训练模型一直以来都困扰着科研人员。以训练 GCN 模型为例，一个节点的表征每次更新都需要递推式地依赖于其邻居节点的表征，这些节点表征之间的运算会随着图的规模以及 GNN 模型隐藏层数的增长呈指数级增长，这会导致过长的训练时间以及大量的内存消耗。GraphSage[6] 在一个 Tesla P100GPU 上需要花费数小时时间才能完成 Amazon 采购网络数据集（包含约二百万个节点）的训练 [7]。而 VRGCN[8] 在一个 Tesla V100GPU（16GB 内存）上训练同样的数据集甚至会出现内存溢出的问题 [9]。为了解决这一问题，科研人员分别引入了采样和分布式计算的方法。通过采样可以在原图上得到一张规模远小于原图的子图，利用采集得到的子图进行模型训练即可达到加速训练的目的，然而采样得到的子图会不可避免地丢失原图的拓扑信息从而使模型的精度下降。另一种方法是将分布式技术应用于图神经网络训练。然而由于图数据的不规律性，很难将传统的分布式机器学习方法直接应用在图神经网络上，需要根据图数据的特性重新设计分布式场景下的任务划分与模型训练模块。如何在分布式场景下高效训练大规模图的图神经网络是一个值得研究的问题。

到目前为止，大部分的 GNN 模型是针对不随时间变化的静态图开发的。然而现实世界中的真实图数据规模往往十分庞大且处于动态变化的状态，例如社交网络、金融交易网络和引文网络。针对动态时序图，如果仍然使用基于静态图的图神经网络算法学习其表征，往往会丢失其时间维度上的特征信息和依赖关系，使得学习到的表征质量不佳。为了解决这一问题，目前主流的解决方

法是采用图神经网络与循环神经网络（Recurrent Neural Networks, RNNs）结合的架构，首先通过 GNN 模型学习图的表征，再输入 RNN 模型中进行时序特征的学习。然而，与 GNN 模型类似，RNN 模型也受到“大规模数据下训练成本过高”这一问题的困扰。在大规模动态时序图上直接应用这种架构学习图表征会进一步凸显这个问题。如何高效地在大规模时序图上进行图表示学习仍是一个值得思考的问题。

1.2 研究问题

本节中首先探讨了现有的基于图神经网络的图表示学习方法的局限，提出大规模时序图表示学习效率问题，并阐述解决该问题所面临的挑战。接着探讨了图神经网络在分布式场景下的训练难点，提出分布式图神经网络训练优化问题，包括数据分区和图计算优化，并阐述解决该问题面临的主要挑战。

1.2.1 大规模时序图表示学习的效率问题

在大规模时序图表示学习任务中，主流的解决方法是使用图神经网络与循环神经网络相结合的框架。首先利用图神经网络学习单时间步上的图表征，接着将这些表征输入循环神经网络中进行训练以学习其时间维度上的特征。然而，无论是图神经网络还是卷积神经网络的训练都涉及大量的矩阵乘积计算，当输入图的规模增大时，其训练时间也呈指数级增长，同时带来极大的计算资源消耗。此外，由于循环神经网络要求输入数据的规模保持不变，而在时序图场景中图中节点可能会产生增减，造成前后时间步之间数据规模的变化，使得循环神经网络无法正常训练。

在大规模时序图中进行图表示学习，同时解决模型训练效率问题，具有如下挑战：

- 如何对动态图进行建模，使得其能刻画图中节点与边随时间变化的动态过程。
- 如何在不过分损失模型精度的情况下尽可能缩短模型的训练时间和节约训练占用的计算资源。

1.2.2 分布式场景下的图神经网络训练问题

分布式图神经网络训练的工作流程一般分为三个阶段：1) 数据划分；2) GNN 模型优化；3) 梯度聚合。其中梯度聚合步骤的过程与传统分布式机器学习方法类似，而前两个过程由于图数据分布的不规律性，在实际实施过程中需要考虑负载均衡，通信开销等具体问题。因此如何在分布式场景下高效训练图神经网络模型具有如下挑战：

- 如何进行数据分区。数据分区是将输入数据（图和特征）分配到一组工作节点中。首先考虑到负载均衡问题，每个工作节点应当接收到规模均匀的子任务从而确保系统不会被某些节点拖慢整体训练速度，而图数据由于节点分布的不规律性，难以像传统数据一样直接进行划分。此外，GNN 模型训练时需要不断聚合目标节点的邻域信息，但在分布式场景中其邻域节点往往会出现本地缺失的现象。如果忽视了这部分缺失的节点，模型的准确性将会被破坏，如果需要获取目标节点的完整邻域信息，则需要向其它机器发送请求，造成大量的通信开销，从而降低分布式训练的效率。
- 如何进行本地 GNN 模型训练。由于数据的依赖性，工作节点进行本地模型训练的方式与传统的深度学习训练方法有很大不同。在不访问其他工作节点获取邻域信息的情况下，可能无法生成正确的计算图从而使得模型的精度下降。如果采取允许访问其他工作节点的模式，又会带来通信协议选择和隐藏层之间参数同步的问题。

1.3 主要贡献

1.3.1 基于图摘要的大规模时序图表示学习算法

本文对大规模时序图场景下的图表示学习进行了研究，在广泛调研现有方法并分析其优劣后，我们提出了一种基于图摘要的大规模时序图表示学习算法——GSAERU。GSAERU 模型包含“图表征学习”与“时序学习”两个模块，将每个时刻上动态图的快照输入到 GSAERU 模型中并生成多个图摘要，并将这些图摘要输入到对应的图自编码器中，利用编码器生成隐藏表征，然后使用解码器将这些表征解码得到原图的预测，并利用与原图的对比得到重构误差，利用此重构误差来训练从而增强隐藏表征的表达能力。在为每个时刻的图中节点都生成对应的表征后，将这些表征输入到一个循环神经网络中训练即可得到最

终的同时包含图结构信息与时序信息的高质量表征。其主要贡献可以总结为以下三点：

- 提出了一种新的图表示学习模型——GSAERU，用于大规模时序图数据上的节点表征学习。现有的采用图神经网络与循环神经网络结合解决动态图问题的图表示学习算法，为了使 GNN 模型训练得到的表征可以满足循环神经网络的输入条件，往往需要固定图中的节点不发生增减变化，而 GSAERU 模型通过引入图摘要技术将原图映射到固定规模的新图中，解决了 GNN 模型输出无法直接输入到 RNN 模型中进行训练的问题。
- 利用图摘要技术加速模型训练和减少模型占用的计算资源。GSAERU 模型对于输入的大规模图，首先利用图摘要技术从其中生成多个规模较小的“子图”，并在这些子图的基础上使用图自编码器学习其表征，并通过将这些图摘要生成的节点表征聚合得到原图的节点表征。由于图摘要技术生成的子图其规模远远小于原图，所以在这些子图上的训练成本也远低于原图，使训练大规模图的速度加快，内存消耗减少。
- 基于四个真实世界中的大规模时序图数据集进行了大量实验。与目前主流的动态图表示学习方法（DynamicTriad、MPNN-LSTM、EvolveGCN 和 TGAT）相比，GSAERU 模型的训练速度更快，消耗内存更少，同时性能与目前最好的算法相比差距不超过 3.5%。

1.3.2 基于图摘要的分布式图神经网络优化训练算法

本文也对分布式图神经网络算法进行了研究，基于充分调研现有分布式图神经网络训练框架的优势与缺陷，我们提出了一种基于图摘要的分布式小批量图神经网络训练框架——DSGNN。DSGNN 由一个领导节点和若干工作者节点构成，其中领导节点使用一个基于边分割的图流划分算法对输入数据进行预处理，其同时负责同步更新 GNN 模型参数。工作者节点则负责利用接收到的划分子图，结合基于图摘要的小批量样本学习方法进行模型的训练，并将结果返回给领导节点。其主要贡献可以总结为以下三点：

- 提出了一种新的基于图摘要的分布式图神经网络训练框架，称为 DSGNN，用于分布式场景下的基于图神经网络的图表示学习。与现有的独立采样分布式图神经网络训练框架不同，该模型可以在工作节点上对子图进行小批量样本的学习，而不是让领导节点完成子图生成。同时，使用图摘要技术

产生的子图，其规模远远小于原图，在其基础上进行模型训练可以大大提升效率。

- 提出了一种新的基于边分割的图流划分算法——Sketch-DBH，用于处理大规模图流数据下的子图划分问题。该算法通过 Count-Min Sketch 这一概率数据结构存储每个节点的度数信息，并使用一个最小堆维护图流数据中的高频节点信息，使其可以在常数时间内返回查询结果并以此根据节点度数信息生成单条图流数据即将被分配到的子图编号。Sketch-DBH 算法可以高效完成大规模图流数据的划分且确保负载均衡，从而加速整个框架的模型训练速度。
- 基于四个真实世界中的大规模图数据集，我们首先使用提出的 DSGNN 框架实现了三个经典图神经网络模型的分布式版本，并比较集中式与分布式训练下模型性能的差异。实验结果表明，与集中式 GNN 模型训练相比，我们的分布式训练算法可以有效缩短训练时间和节约计算资源，且绝大部分情况下分布式与集中式训练的结果在 F1 值这一指标上相差不超过 1%。接着将提出的框架与三个主流的分布式图神经网络训练框架（ROC、DistDGL 和 Aligraph）进行对比，比较其在相同数据集和 GNN 模型下的性能差异。结果表明 DSGNN 对分布式 GNN 模型训练的提升效果显著高于其它三种方法。最后我们通过数据可视化显示了在 DSGNN 框架下训练的 GNN 模型的鲁棒性。

1.4 论文结构

本文主要研究了大规模时序图场景下的图表示学习方法。经过广泛调研现有的动态图表示学习方法与神经网络训练加速技术并分析其优点与不足后，我们提出了基于图摘要的大规模时序图表示学习方法与分布式图神经网络训练框架。因此，本文的结构如下：

- 第一章：简要阐明了本文的研究背景，解释了研究问题并列举了本文的主要贡献。
- 第二章：广泛调研了目前的图表示学习方法，以及针对大规模时序图上图表示学习的优化方法，并对这些现有方法分析其优点与不足，之后进行总结分类。
- 第三章：详细介绍了我们提出的基于图摘要的大规模时序图表示学习算

法。首先解释了该方法的研究动机，接着对时序图问题进行建模，然后介绍了我们方法的详细架构，最后通过在四个大规模图数据集上进行的大量实验，证明了我们方法的优越性。

- 第四章：详细介绍了我们提出的基于图摘要的分布式图神经网络训练框架。首先基于对现有分布式图神经网络训练框架的分析，给出了问题的定义与建模，然后详细介绍了我们方法的架构，最后通过在四个真实的大规模图数据集上与其它框架进行性能对比，论证了我们方法的优越性。
- 第五章：系统总结了本文的工作与贡献，并探讨了图表示学习未来的研究方向。

第二章 相关工作

图神经网络一直是解决图表示学习问题的有效方法，在社交网络、生物医疗、交通规划等领域都取得了成功的应用。本章结合现有研究，将图神经网络分为三类：（1）基于静态图的方法；（2）基于动态图的方法；（3）基于分布式的方法。之后将按照分类对相关领域的研究进行文献综述。最后，本章将介绍一种针对大规模图数据的降维方法——图摘要。图摘要技术通过聚合，简化等方法在原图的基础上生成一个较小的摘要图，该摘要图能保存原图中的有效信息，从而大大减少存储代价并加快信息提取分析速度。

2.1 图神经网络

随着神经网络和深度学习技术的广泛应用，受到卷积神经网络和循环神经网络成功案例的启发，科研人员试图将神经网络技术推广到图这一领域中，并产出了许多优秀的成果。本节将目前最火热的图神经网络算法根据应用场景的不同总结为三类——基于静态图的方法、基于动态图的方法和基于分布式的方法。接下来将对这三类方法分别进行介绍和总结。

2.1.1 基于静态图的方法

针对静态图的图神经网络设计，研究人员通常使用图卷积、注意力机制和图采样等方法来让图中的节点学习图的拓扑信息以及其邻域的节点特征信息，从而获得高质量的表征向量。

目前，图卷积神经网络（Graph Convolutional Network, GCN）[10] 模型被普遍认为是进行图表示学习的有效方法。GCN 是一种端到端的半监督学习方法，适用于具有任意大小和形状的图结构数据。GCN 使用非欧几里得域中的图拉普拉斯算子 [11] 进行卷积运算并迭代聚合节点的邻居嵌入，以此模仿传统卷积神经网络中的卷积操作，从而使图表征学习到图拓扑信息和节点结构信息。

其使用的前向传播规则为:

$$f(H^l, A) = \sigma(D^{-\frac{1}{2}} \hat{A} D^{-\frac{1}{2}} H^l W^l) \quad (2-1)$$

其中 A 为图的邻接矩阵, $\hat{A} = A + I$, I 为单位矩阵, D 为图的度矩阵。

此外, 在 GCN 的基础之上, Derr 等人提出了带标签的图卷积神经网络 (SGCN) [12], 通过节点附带的标签信息分别设置了正负标签的传播规则, 从而解决了图中正负标签节点分布数量不平衡影响表征表达效果的问题。Kingma 等人提出了基于 GCN 的变分图自编码器 (VGAE) [13], 其采用“编码器-解码器”的结构, 首先由作为编码器的 GCN 将输入数据映射到一个隐藏表示空间得到隐藏表征, 接着利用解码器将隐藏表征还原到原图空间, 通过最小化输入和输出的重构误差使得隐藏表征学习到原图的结构信息, 从而得到高质量的节点表示。

另一种主流的方法是将注意力机制与图神经网络结合, 利用注意力机制使得模型在图表示学习过程中更关注与训练任务更相关的图的部分。这种处理的优点在于可以降低图中“噪声”对训练结果的影响, 从而增加信息处理中的信噪比 [14]。基于注意力机制的图神经网络旨在为目标节点 t 的邻域节点分配注意力权重 $\alpha_i \in [0, 1]$, 且其满足 $\sum_{i \in N(t)} \alpha_i = 1$, $N(t)$ 为节点 t 的一阶邻居集合。velivc 等人提出的图注意力网络 (Graph Attention Network, GAT) [15] 使用隐藏的自我注意力层来学习节点表征, 其目的是计算注意力系数:

$$\alpha_{ij} = \frac{\exp(\text{LeakyReLU}(a[Wh_i || Wh_j]))}{\sum_{k \in N(i)} \exp(\text{LeakyReLU}(a[Wh_i || Wh_k]))} \quad (2-2)$$

在 GAT 的基础之上, ABU 等人提出了基于图注意力机制的随机游走算法 (AttentionWalks) [16], 其从目标节点开始采用随机游走策略采集邻域信息, 并使用图注意力系数确定下一步将要访问的节点。XuQ 等人将注意力机制应用于图递归神经网络, 使模型专注于保存更多相关语义信息的节点 [17]。它通过对连续 k 个相邻节点进行采样并生成子图以构建递归神经网络, 并通过注意力系数来控制邻域信息传递到目标节点的权重。

尽管注意力机制已被证明可以用来提高某些图表示学习模型的性能, 但它并不是总能提高图嵌入的准确性 [18]。

除了采用图卷积和图注意力机制, 科研人员还尝试了许多其它方法来学习图表征。Hamilton 等人提出了使用采样和聚合的方法归纳学习节点嵌入

(GraphSage) [19]。其训练了一组聚合函数，首先利用目标节点邻域节点的特征信息进行学习，之后将这些局部领域的特征分别聚合并传递给目标节点。Wang 等人提出了一种基于对抗生成训练的图表示学习算法 (GraphGAN) [20]。其借鉴了生成对抗网络 (GAN) 的思想，为了拟合目标节点邻域的真实分布，GraphGAN 使用生成器对图中节点之间的连接概率进行建模，并生成虚拟节点，再使用鉴别器评估目标节点与其邻域节点之间有边的概率，以区分生成器生成的虚拟节点和真实存在的邻域节点。通过生成器和鉴别器之间的对抗提升图表征的鲁棒性。

2.1.2 基于动态图的方法

近几年来，针对时序图上的动态图表示学习问题引起了科研人员的兴趣。现有的针对时序图的表示学习方法大多是由静态图上的方法拓展而来。

一种流行的动态图表示学习方法是将动态图中节点和边随时间的变化建模为连续的点过程 (stochastic processes) [21]。KnowEvolve[22] 和 DyRep[21] 将边随时间的增减建模为点过程函数，并使用神经网络将点过程函数参数化，从而对该边连接的节点嵌入进行更新。DynamicTriad[23] 使用点过程模拟动态图中更复杂的变化模式——三元闭包过程，其描述了图中具有三个节点的三元组从一对节点不连接的状态发展到任意节点两两连接的状态。模型基于所有的三元闭包过程学习图中节点的表示。HTNE[24] 同样对节点的点过程进行建模，另外加入了注意力机制来学习邻居节点对当前节点长期和短期的影响，最终根据该注意力系数更新目标节点表征。这类方法通过将动态图的随机变化过程建模为以时间为参数的连续点过程函数，使得模型能够有效生成动态图的表示并以此预测动态图的变化。

得益于循环神经网络 (Recurrent Neural Networks, RNNs) 的成功，另一种对动态图表示进行学习的方法是在图神经网络中引入 RNN 架构。TGN[25] 和 JODIE[26] 仿照 LSTM 的结构为每个图节点维护一个记忆单元和隐藏单元，并通过节点随时间变化的情况更新这些单元，最后利用隐藏单元训练得到节点的表示。EvolveGCN[27] 提出使用 RNN 辅助训练 GCN 模型的参数。其将动态图建模为一个静态图序列，序列中的每张图代表其在对应时刻的图快照。接着构建一个标准 LSTM 模型，并将其中隐藏单元的全连接层替换为 GCN 模型的图卷积层，之后将这些静态图的邻接矩阵与特征矩阵输入 LSTM 模型，从而达到更新 RNN 隐藏状态的同时训练 GCN 模型的目的。尽管基于 RNN 的动态图

表示学习方法可以有效保存和学习与时间维度相关的特征和事件并预测图的变化趋势，但由于 RNN 本就高昂的训练成本，结合 GNN 模型后需要进行大量的高维向量运算，会造成大量的内存占用，产生极大的计算成本。

2.1.3 基于分布式的方法

利用图神经网络进行图表示学习可以得到高质量的表示，但随着图规模的增大，其消耗的训练时间也急速增长。为了达到加速训练的目的，科研人员尝试将分布式技术与图神经网络结合，并产出了许多优秀的成果，目前主流的分布式图神经网络算法可以分为以下四类：

分布式全批量训练中分派子图方法：此类方法在每轮训练中对完整原图进行训练。首先将图数据和待训练的 GNN 模型输入领导节点，在训练过程中领导节点根据具体的节点特征聚合任务分发给各个工作者节点进行计算，然后收集返回结果并根据损失函数计算梯度，进行模型参数更新。NeuGraph[28] 通过将每个顶点的特征平均分成多个部分来探索特征级并行性，从而以更细粒度的方式划分工作负载。然后将其分配给不同的 GPU 线程进行相应的计算。该方法充分利用了 GPU 海量的并行计算资源。ROC[29] 将 GPU 内存管理转化为一个成本最小化问题，它使用递归动态规划算法寻找全局最优解来决定每一块数据应该缓存在哪个 GPU 内存中以供重用。然而，由于只有领导节点保存模型参数且训练时严重依赖于工作者节点的计算结果，导致这类方法的数据通信开销往往很大，且每轮训练的运行时间呈现不规律性。

分布式全批量预处理子图划分方法：此类方法也是在每轮训练中使用对完整原图进行训练，在得到输入的图数据和 GNN 模型后，增加了预处理阶段使领导节点将图提前进行划分，并将每个划分后的子图和一份 GNN 模型的副本发送给对应的工作者节点，由它们在本地进行 GNN 模型的训练，最后将结果返回领导节点，由领导节点汇总后再统一通知工作者节点更新模型参数。DGCL[30] 分布式 GNN 训练提供了一个通用且高效的通信库。它将最短路径生成树算法运用到传输规划中，通过“避免竞争”的策略提高链路使用效率并平衡不同链路上的工作负载。FlexGraph[31] 通过合并位于同一分区的相邻顶点的特征以减少数据传输量。当每个计算节点收到来自其他节点的邻居信息请求时，首先会在本地聚合部分邻居信息，然后将部分结果发送给请求计算节点，而不是直接发送初始邻居信息。请求计算节点只需要将接收到的部分结果与其本地节点进行聚合即可继续计算，这使得通信开销显著减少。然而，这类设计

的好处是最大限度地调用了每个工作者节点的计算资源，缺点是在模型训练中工作者节点需要通过网络通信向其它节点获取其子图上待训练节点缺失的邻域信息，造成大量的通信开销。

分布式小批量独立采样方法：此类方法首先由领导节点对输入图数据进行采样，并将采样得到的子图和输入 GNN 模型的副本分别发送给工作者节点，由工作者节点在本地根据子图进行 GNN 模型训练。梯度计算和模型参数更新仍是由领导节点来完成。AGL[32] 提出了一种分布式管道，根据消息传递生成每个顶点的 k 跳邻域，并使用 MapReduce 框架实现它。生成的 k 跳邻域信息存储在分布式文件系统中。由于只需要在生成小批量样本时收集目标节点的 k 跳邻域信息，这样的设计大大加快了采样过程。SALIENT[33] 通过使用 CPU 的多线程技术并行化小批量样本生成。它使用 C++ 的多线程技术端到端地实现小批量样本生成，从而提高了采样的性能。相较于前两类方法，本方法能够大大提升训练速度，但整体模型的运行效率仍由领导节点的采样效率来决定。

分布式小批量联合采样方法：此类方法中领导节点不再对原图采样，而仅仅只是划分原图，并将划分子图和 GNN 模型副本发送给工作者节点。之后由工作者节点在得到的划分子图上进行采样并将采样结果作为本地 GNN 模型的输入进行模型训练。梯度计算和模型参数更新仍是由领导节点完成。AliGraph[34] 在其系统中实现了四种不同的图划分算法，其能适用于大部分类型的图数据输入。2PGraph[35] 提出了 GNN 层次感知缓存。它采用固定顺序对目标节点进行采样，这使得每个节点的训练顺序是固定的。通过在计算之前将所有目标顶点的 k 跳邻居预先缓存到 GPU 内存中，几乎可以完全消除数据传输的时间开销。这类方法的性能瓶颈在于领导节点的原图划分效率和工作者节点上的采样训练效率。

2.2 图摘要

图是一种现实世界上最常见的数据结构，包括社交网络、交通网络、计算机网络、电话呼叫或消息传递网络、位置签到网络、蛋白质-蛋白质相互作用网络、用户-产品评论或购买网络、功能或结构脑连接组等等。随着技术的进步，我们生成的数据量以及我们收集和存档此类数据的能力都在不断增加，大量生成的数据以及极快的数据产生速度对及时处理提出了挑战。图摘要技术作为解决大规模图存储与分析的有效手段，目前受到了越来越多研究人员的关注。图

摘要算法以大规模的图作为输入，经过对结构信息和特征信息的提炼，输出一个小的摘要图，摘要图由超级节点和超边组成，富含原图中的特征信息 [36]。根据使用技术的不同可以将其分为以下三类——基于位压缩的方法、基于简化的方法和基于 sketch 的方法。下面将对其进行一一介绍并总结。

2.2.1 基于位压缩的方法

位压缩是数据挖掘中的常用技术。在图摘要算法中，这类方法的目标是最小化描述输入图所需的位数，其中图摘要由输入图及其未建模部分的模型组成。图摘要与模型的规模均远小于原图，且能表达原图的结构模式，如二分图。具体的处理方法有压缩和二次分组技术，这类方法通常将图摘要任务定义为一个模型选择任务，通过最小描述长度（Minimum Description Length, MDL）编码将原图 G 和模型 M 转换为二进制位串，并通过最小化这两个二进制位串之间的距离来确定模型 M 的类别：

$$\arg \min_M L(G, M) = L(M) + L(G|M) \quad (2-3)$$

其中 $L(M)$ 为模型 M 的描述长度， $L(G|M)$ 为给定模型下原图的描述长度。

Navlakha 等人 [37] 提出了一种基于有限误差的图摘要方法。其首先通过 MDL 将节点聚合从而得到图摘要，并另外维护一个校正边集合。该方法生成的摘要图是一个聚合图，每个节点对应原图中的一组节点，并可以借助校正边集合查询到正确的原图结构。

2.2.2 基于简化的方法

基于简化的图摘要方法通过删除不太“重要”的节点或边来简化原始图，从而产生稀疏图。与生成“超图”的图摘要方法相反，此类方法生成的摘要图由原始节点与边的子集组成。

基于节点简化的摘要技术的代表性工作是 OntoVis[38]，用于简化大型异构社交网络。OntoVis 使用本体中与节点和边相关的信息（例如特定类型节点的度数）来语义修剪网络。其支持语义抽象、结构抽象和重要性过滤三种摘要生成方法。在语义抽象中，用户通过仅包括其类型从原始本体图中选择的节点来从原始图构造派生图。而结构抽象简化了图，同时保留了整个网络的基本结构。重要性过滤利用节点度等统计指标来评估节点类型之间的连通性和相

关性。

2.2.3 基于 sketch 的方法

sketch 是一个概率数据结构，可以高效地处理大规模数据的计数问题，其中最成功的应用为 Count-Min Sketch。如图 2-1 所示，Count-Min Sketch 由 d 个哈希函数构成，其结构可以用矩阵表示，矩阵的每一行分别代表一个哈希函数。每个元素通过 d 个哈希函数映射到矩阵每一行中的一个单元格上。每当该元素的值更新时，通过哈希表查询每一行其存储的位置并将这些位置的值均加上更新值。在查询阶段，根据 d 个哈希查询到 d 个结果，并取其中的最小值作为结果返回，利用这种方法的查询精度为：

$$P(c_x \leq \hat{c}_x \leq c_x + \varepsilon n) \geq 1 - \delta \quad (2-4)$$

其中 n 为数据流大小， c_x 和 $\hat{c}_x$ 分别为真实值和预测值， ε 与 δ 为自定义值，只需令 Count-Min Sketch 的行数和列数分别等于 $\lceil \frac{n}{\varepsilon} \rceil$ 和 $\lceil \ln \frac{1}{\delta} \rceil$ 上式即可成立。

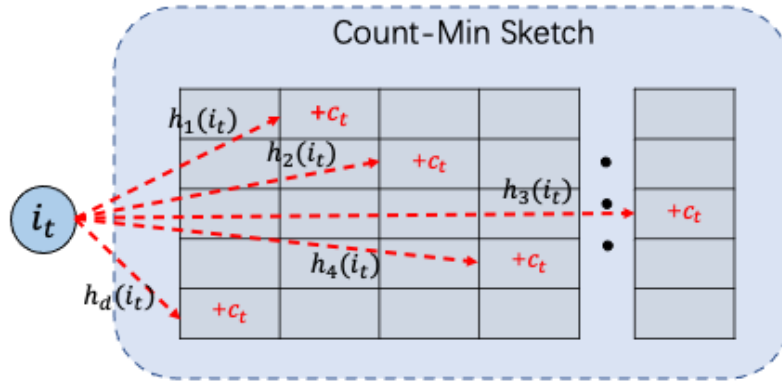


图 2-1: Count-Min Sketch 数据结构原理示意图

基于 sketch 用哈希映射的方法压缩数据，Tang 等人提出了基于图 sketch 的图摘要算法（TCM）[39]。该算法通过创建 d 个图 sketch 并返回联合查询结果来近似各种图查询。每个节点都是通过一个哈希函数映射到 d 个图 sketch 的超级节点上，图 sketch 中超节点之间的边对应于它们的组成节点之间的连接总和的超边。

第三章 基于图摘要的大规模时序 图表示学习算法

3.1 研究动机

近年来，随着图表示学习（Graph Representation Learning）在社会科学、生物学、物理学等领域的问题上取得了一系列成就，图神经网络（Graph Neural Networks, GNNs）已成为了一个热门的研究课题。其主要通过深度学习技术学习非欧几里得数据——图的拓扑结构与特征信息，并为图中的节点与边生成对应的高质量表征，从而应用于链路预测、节点分类、图分类等下游任务。它在网络流量预测、金融风险评估、社交网络信息传播等现实领域有着广泛的应用前景。针对图表示学习的研究，科研人员已设计了许多经典且有效的图神经网络结构 [6, 10, 15]。然而在实际应用中还存着两个巨大的挑战。

第一，到目前为止，大部分的 GNN 模型是针对不随时间变化的静态图开发的。然而，现实世界中的很多图是随时间的推移而动态变化的，例如社交网络、金融交易网络和引文网络。图 3-1 展示了一个用户与博文互动并相互关注的动态网络，其中的每条边都有一个时间戳，这些动态行为传达了重要的信息，可以挖掘出用户在时间维度上的依赖关系，帮助我们更好地预测用户未来的行为。但如果只考虑静态图，这些信息就会丢失。

第二，尽管 GNN 模型在处理图数据上展现出了优秀的能力，但如何高效地大规模图数据上训练模型一直以来都困扰着科研人员。以训练 GCN 模型为例，一个节点的表征每次更新都需要递推式地依赖于其邻居节点的表征，这些节点表征之间的运算会随着图的规模以及 GNN 模型隐藏层数的增长呈指数级增长，这会导致过长的训练时间以及大量的内存消耗。GraphSage[6] 在一个 Tesla P100GPU 上需要花费数小时时间才能完成 Amazon 采购网络数据集（包含约二百万个节点）的训练 [7]。而 VRGCN[8] 在一个 Tesla V100GPU（16GB 内存）上训练同样的数据集甚至会出现内存溢出的问题 [9]。

针对第一个挑战，研究人员尝试将时间维度上的特征更新纳入模型中从而

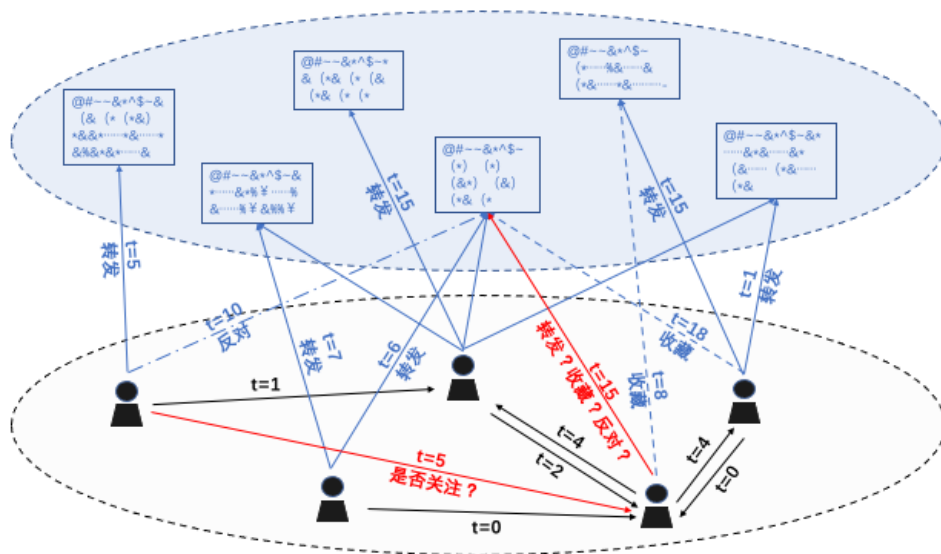


图 3-1: 真实场景中用户与博文互动并相互关注的动态网络

将静态图方法扩展为时序图方法。DANE[40] 基于矩阵分解的方法，利用图拉普拉斯矩阵的特征向量计算图节点的嵌入，并根据矩阵随时间的变化动态地更新特征。然而这类基于矩阵分解的算法只关注了图中的结构拓扑信息，无法将节点的特征信息纳入考量。HTNE[24] 将图中边随时间的变化建模为一个连续的点过程，并使用注意力机制来衡量历史邻居对节点当前邻居的影响，最后利用神经网络进行训练。然而这类方法只关注了时序图中边的变化情况，忽视了节点特征也会随时间变化的问题。随着循环神经网络（Recurrent Neural Networks, RNNs）的兴起，使用 GNN 与 RNN 相结合的架构成了目前研究时序图问题的主流方法。如图 3-2 所示，这类方法使用 GCN 或 GAT 获取节点嵌入，然后将其输入到学习时序性的 RNN 模型中，这样就能获得兼顾图信息与时序信息的表征，GCRN[41] 和 TGAT[42] 是其中的代表方法。然而为了保证 RNN 模型训练的性能，这类方法往往需求时序图中的节点不会发生变化，即每个节点在每个时间步上都必须出现。同时由于 RNN 的训练也非常耗时，因此图的规模成为了制约此类模型训练的瓶颈。

针对第二个挑战，研究人员开发了许多加速 GNN 模型训练的方法。其中最成功的一类方法是在原图上采样一些很小的子图，并利用采样生成的子图进行小批量训练，从而达到加速训练和减少内存消耗的目的 [7, 8, 43, 44]。RN 采样 [45] 使用一个均匀随机选择方法进行采样，RPN 采样 [46] 借助经典的

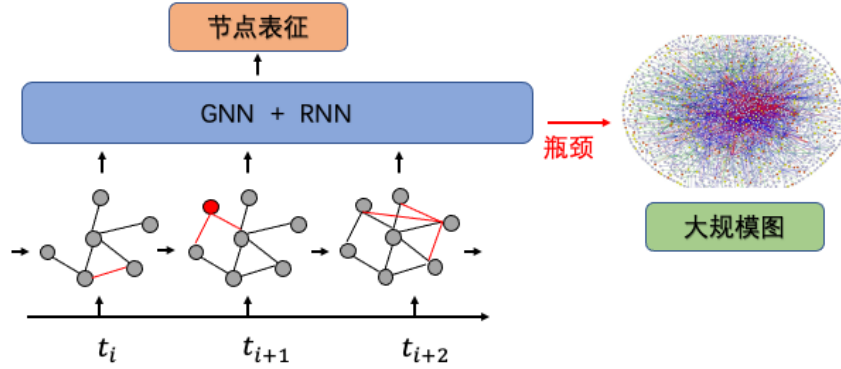


图 3-2: 现有时序图预测方法原理及瓶颈

PageRank 算法辅助进行采样, RW 采样 [47] 通过在图中的路径上进行随机游走完成采样, Snow-Ball 采样 [48] 使用广度优先搜索从一个随机起点开始采样。然而这些采样子图会不可避免地丢失原图的拓扑信息, 为了避免这样的情况发生, 就不得不增加每轮训练采样子图的数量。但随着采样子图的数量增大, 又会给模型的训练增加负担。如何在不过分降低模型精度的前提下加速 GNN 模型训练仍是一个值得研究的问题。

为了解决在大规模时序图上进行图神经网络快速训练和预测的实际挑战, 本章节提出了一种新的图表征学习与时序预测模型, 称为 GSAERU, 用于在大规模时序图上进行表征学习和预测。GSAERU 模型由图表征学习和图时序学习两个模块组成。在图表征学习模块, 采用图摘要 (graph sketch) 技术将时序图中每个时间步上的快照“压缩”成一组规模远远小于原图的图, 并输入到一组图自编码器 (Graph Auto-Encoder, GAE) 中进行学习, 得到低维高质量表征, 达到加速训练大规模图的目标。在图时序学习模块, 采用循环神经网络训练图表征学习模块得到的表征, 并输出预测结果, 完成在大规模时序图上的训练。基于在真实世界中的大规模时序图数据集上的大量实验, GSAERU 在加速训练图神经网络训练和维持模型精度方面取得了突出的成绩。

3.2 问题建模

如图 3-3 所示, 动态时序图可以用一组静态图序列 $\{\mathcal{G}^1, \mathcal{G}^2, \dots, \mathcal{G}^T\}$ 表示, 其中 $\mathcal{G}^t = (A^t, X^t)$ 来表示序列图在时刻 $t (1 \leq t \leq T)$ 时的快照, $A^t \in R^{n_t \times n_t}$ 为序列图在时刻 t 时的邻接矩阵 (n_t 可能随时间的变化而发生改变), $X^t \in R^{n_t \times d}$ 为序

列图在时刻 t 时的特征矩阵，维度为 d （真实场景中节点的特征会随时间发生变化，但特征维度一般固定，故设为定值）。

给定一个时序图序列 $\{\mathcal{G}^i, \mathcal{G}^{i+1}, \dots, \mathcal{G}^{i+T}\}$ ，目标是为时序图中的每个节点生成一个包含其结构信息、特征信息、时序信息的表征 $z \in R^{1 \times c} (c \ll d)$ ，并利用这些表征预测 $i+T+1$ 时刻时序图的变化，即预测 $\mathcal{G}^{i+T+1}$ 。

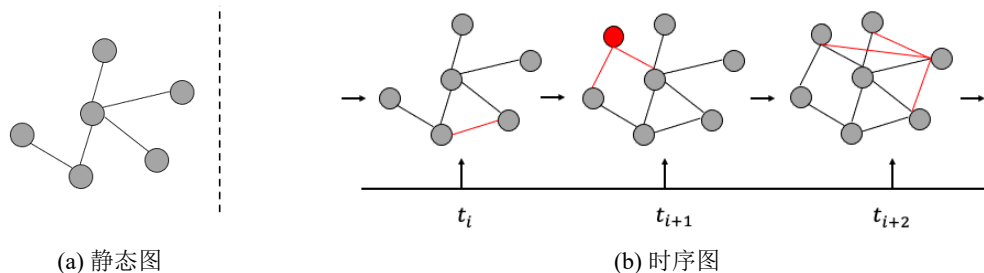


图 3-3: 静态图与时序图的对比示例

3.3 模型设计和算法描述

本章节提出了使用 GSAERU 模型加速训练大规模时序图的机制，首先介绍了模型整体框架，接着分为两个模块详细介绍了 GSAERU 模型的结构与训练方法，最后介绍了如何使用模型预测时序图变化趋势。

3.3.1 模型框架

基于图摘要加速的大规模时序图预测模型的整体框架如图 3-4 所示，它由图表征学习和图时序学习两个模块组成。在图表征学习模块，GSAERU 模型首先在时序图的快照上生成一组图摘要，并分别输入到一组图自编码器中进行训练，利用学习得到的低维表征即可通过聚合等操作获得时序图上的节点表征。在图时序学习模块，GSAERU 模型通过将上一步生成的图摘要的表征进行拼接，得到时序图在时间维度上的特征，并输入到循环神经网络中进行训练，从而预测下一时刻时序图的拓扑信息与节点特征信息。

3.3.2 图表征学习模块

为了在图上进行链路预测、节点分类等任务，往往需要先通过图表征学习得到图中每个节点的高质量表征。在本节中我们将介绍如何使用基于 sketch 的

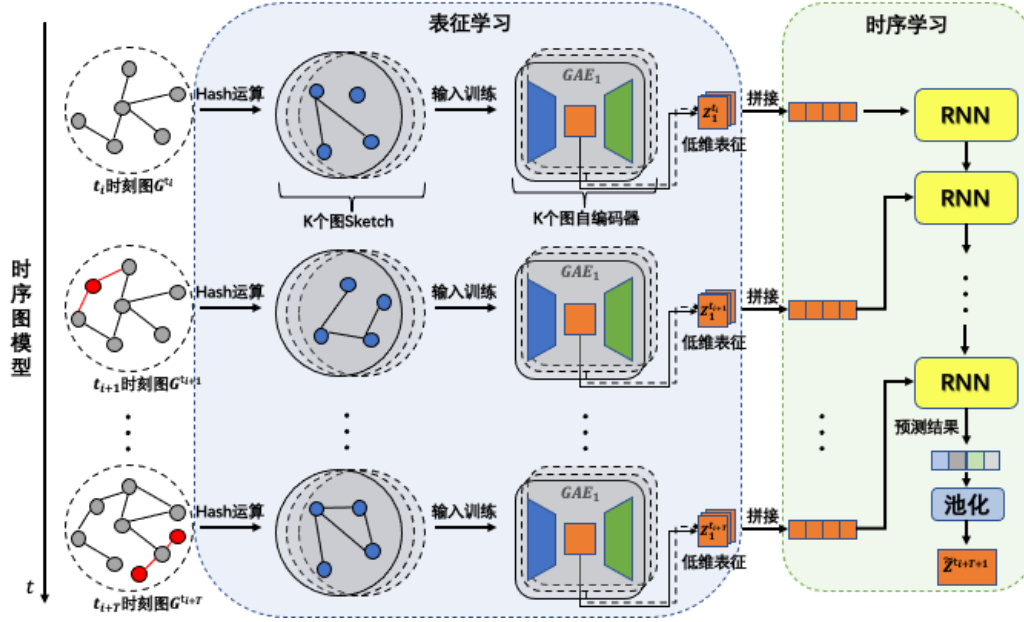


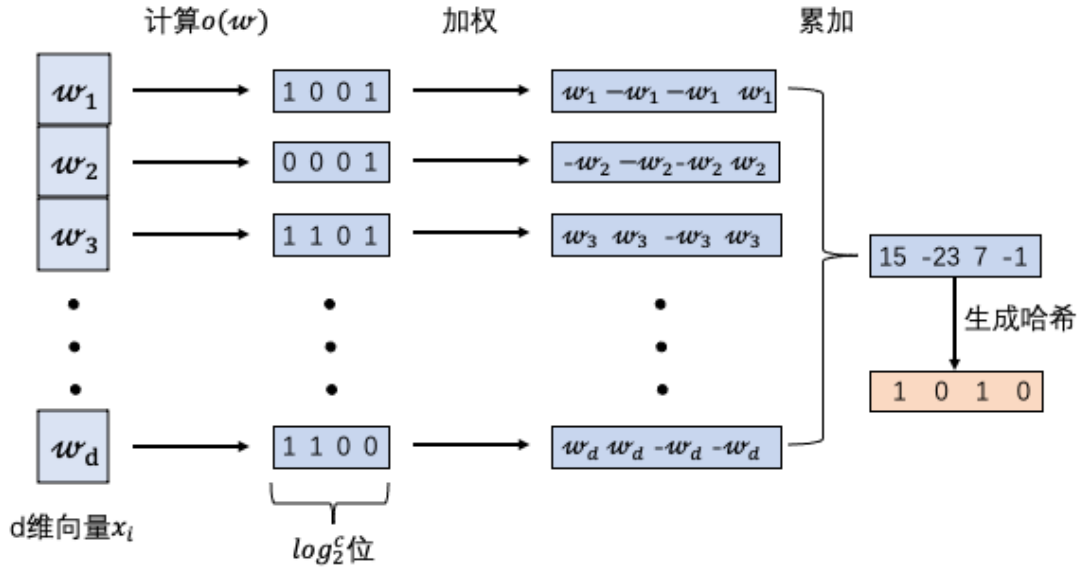
图 3-4: GSAERU 模型框架

图摘要技术对原图进行采样和利用图自编码器模型得到节点的低维高质量表征。同时，现有的基于 GNN 结合 RNN 的方法为了满足 RNN 模型训练的输入格式，往往要求图中图的节点数不能随着时间推移而改变（即每个节点都必须出现在所有的时间步中），而 GSAERU 中的图表征学习模块则能针对节点变化这一情况，学习到固定规模的表征向量，为下一节的图时序学习创造便利条件。

3.3.2.1 图摘要生成

第 2.2.3 节中我们介绍了基于 sketch 的图摘要技术的背景知识，本节将主要介绍 GSAERU 模型如何为时序图中一个时刻的快照 $\mathcal{G}^t$ 生成对应的图摘要： $\mathcal{GS}^t$ 。

为了生成相应的图摘要，GSAERU 模型通过一个哈希函数 $h(\cdot) : R^d \rightarrow [c]$ 将原图中的节点映射到一个固定节点规模为 c 的新图中，其中 d 为原图中节点的特征维度， c 为哈希桶的大小。更具体的，模型采用 SimHash[49] 方法进行图 Sketch 的构建。SimHash 是一种局部敏感的哈希算法，它能使在原来空间相似的样本集合，通过相关运算映射到特定范围空间时仍然是相似的，反之还能保证原来不相似的经过哈希后极大概率不相似，这一特性也符合图的同质性假说[50]。

图 3-5: SimHash 方法计算 $h(v_i)$ 流程示例

SimHash 方法计算哈希值 $h(\cdot)$ 的过程如图 3-5 所示。以图 $\mathcal{G}$ 上的一个节点 v_i 为例，其原始特征为 d 维向量 x_i 。第一步，选择一个基础哈希函数 $o(\cdot) : R^d \rightarrow [c]$ (c 为固定常数且远小于 n_t)。第二步，使用 $o(\cdot)$ 计算向量 x_i 每个特征的哈希值，并将结果转化为二进制进行存储。第三步，对这 d 个二进制数进行加权处理：如果某位为 1，则将该位的值设为其对应的特征值 w ，否则置为 $-w$ 。第四步，将加权处理后的所有向量进行累加，并判断累加结果上每一位的正负。若为正，则将该位置为 1，否则置为 0。最后得到的二进制数即是 SimHash 的哈希值 $h(v_i)$ 。容易计算 $h(v_i)$ 的时间复杂度为 $O(d \log_2 c)$ 。为了得到图 Sketch 的邻接矩阵 AS ，首先生成一个规模为 $c \times c$ 的零矩阵 AS ，然后对原图中的每条边 (v_i, v_j) 分别计算两个节点的哈希值，并将新矩阵的对应位置的值加一。最后根据公式 3-1 得到图 Sketch 的邻接矩阵，其中 α_{GS} 为实验设置的超参数。易得计算邻接矩阵时间复杂度为 $O(|E|d \log_2 c)$ 。

$$as[i][j] = \begin{cases} 0 & as[i][j] < \alpha_{GS} \\ 1 & as[i][j] \geq \alpha_{GS} \end{cases} \quad (3-1)$$

接着对原图中每个节点进行哈希运算从而得到它们对应的哈希桶的编号，如 $h(v_i) = 2$ ，表明原图中的节点 v_i 映射到了新图中编号为 2 的节点上，同时维护一个长度为 c 的向量 $value$ ，用于记录每个哈希桶中节点的数量，如 $value[2]$ 表示映射到新图编号为 2 上的原图节点数量。令新图中节点的特征为映射到该

节点的原图节点特征的聚合：

$$xs_j = \frac{\sum_{h(v_i)=j} x_i}{value[j]} \quad (3-2)$$

通过上式可以以 $O(nd \log_2 c)$ 的时间复杂度完成图 Sketch 中特征矩阵 XS 的构建。至此我们通过 SimHash 完成了一个图 Sketch 的构建。

GSAERU 模型采用了 k 个两两相互独立的 SimHash 函数 $\{h_i(v)|i = 1, \dots, k\}$ 为每个时刻的时序图生成了 k 个图 Sketch $\{\mathcal{GS}_i|i = 1, \dots, k\}$ 。通过对这 k 个图 Sketch 进行联合查询即可得到原图正确的拓扑信息 [39]。因此我们可以将对原图训练转化为对这 k 个图 Sketch 进行训练，由于每个图 Sketch 的规模都远远小于原图，所以训练图 Sketch 需要的时间和运算资源均远远少于对原大规模图的训练，从而达到加速训练的目的。

3.3.2.2 图自编码器

在上一节中，我们针对时序图 $\mathcal{G}$ 在时刻 t 的快照 $\mathcal{G}^t$ 生成了 k 个图 Sketch： $\{\mathcal{GS}_1^t, \mathcal{GS}_2^t, \dots, \mathcal{GS}_k^t\}$ ，并通过在这些图 Sketch 上的联合查询可以以高准确度来还原原图。这表明可以通过学习这些图 Sketch 上的结构和特征信息得到表征，再通过聚合这些图 Sketch 的表征得到包含正确图结构信息的原图上节点的高质量表征。故本节我们采用图自编码器（Graph Auto-Encoder, GAE）来学习这些图 Sketch 上节点的表征。图自编码器是一种学习图节点表征的无监督神经网络模型，它由一个编码器和一个解码器构成，通过编码器将输入数据“压缩”为低维特征，再通过解码器将这低维特征重新还原为原始数据。通过最小化这一重构误差使得解码器的输出尽可能近似输入数据，从而使得模型能够学习到输入数据的隐藏特征。

在 GSAERU 模型中，由于每个时间步生成的图 Sketch 数量固定为 k 个，所以在每个时间步中需要将这 k 个图 Sketch 分别输入到 k 个 GAE 中进行训练。以时刻 t 生成的一个图 Sketch—— $\mathcal{GS}_1^t = (AS_1^t, XS_1^t)$ 为例，首先通过编码器得到一个低维表征矩阵 $Z_1^t \in R^{c \times b} (b \ll d)$ 。GSAERU 模型采用的编码器为一个包含两个隐藏层的 GCN，低维特征矩阵的计算如下：

$$\begin{aligned} Z_1^t &= GCN_1(AS_1^t, XS_1^t) \\ &= \widetilde{AS_1^t} ReLU(AS_1^t XS_1^t W_{1,0}) W_{1,1} \end{aligned} \quad (3-3)$$

其中 $\widetilde{AS}_1^t = DS_1^{t-\frac{1}{2}}AS_1^tDS_1^{t-\frac{1}{2}}$, DS_1^t 为 $\mathcal{GS}_1^t$ 的度矩阵, $W_{1,0}$ 和 $W_{1,1}$ 为待学习的参数矩阵。

接下来介绍解码器部分, GSAERU 模型使用两层多层感知机对 Z_1^t 进行解码。值得注意的是, 与传统 GAE 用解码器还原输入数据不同, GSAERU 模型使用解码器还原的不是输入的 $\mathcal{GS}_1^t$, 而是当前时刻时序图的邻接矩阵 A^t 。以原图 $\mathcal{G}$ 中的两个节点 v_i 与 v_j 为例, 首先通过哈希函数 $h_1(\cdot)$ 确定它们在图 Sketch 中对应的表征向量 $z_{1,h_1(v_i)}^t$ 与 $z_{1,h_1(v_j)}^t$, 这两个向量分别代表 Z_1^t 的第 $h_1(v_i)$ 行和 $h_1(v_j)$ 行。接着通过拼接操作 ($\parallel$) 将这两个向量拼接并输入到解码器中, 通过解码器计算它们之间在当前时间步存在边的概率 $\hat{a}_{ij}^t$:

$$\begin{aligned}\hat{a}_{ij}^t &= \frac{MLP_1(z_{1,h_1(v_i)}^t \parallel z_{1,h_1(v_j)}^t) + MLP_1(z_{1,h_1(v_j)}^t \parallel z_{1,h_1(v_i)}^t)}{2} \\ &= \sigma\left(\frac{W_{1,3}(W_{1,2}(z_{1,h_1(v_i)}^t \parallel z_{1,h_1(v_j)}^t)) + W_{1,3}(W_{1,2}(z_{1,h_1(v_j)}^t \parallel z_{1,h_1(v_i)}^t))}{2}\right)\end{aligned}\quad (3-4)$$

其中, $W_{1,2}$ 和 $W_{1,3}$ 为待学习的参数矩阵, $\sigma(\cdot)$ 为 Sigmoid 函数, 用于将数值归约到 (0, 1) 上。

通过这样的设计, 就可以重构时序图的邻接矩阵 $\hat{A}^t = \{\hat{a}_{ij}^t | 1 \leq i, j \leq n_t\}$, 继而计算重构误差。在 GAE 的训练过程中, 采用交叉熵作为损失函数:

$$\mathcal{L}_{rec} = -\frac{1}{n_t^2} \sum_{i=1}^{n_t} \left[\sum_{j=1}^{n_t} (a_{ij}^t \log \hat{a}_{ij}^t + (1 - a_{ij}^t) \log(1 - \hat{a}_{ij}^t)) \right] \quad (3-5)$$

从损失函数可以看出, 重构的邻接矩阵 $\hat{A}^t$ 与原时序图的邻接矩阵 A^t 越接近, 结果越好。之所以使用解码器重构原图而不是原图生成的图 Sketch, 是因为图 Sketch 中本身就包含了一部分原图的正确结构信息, 我们希望通过 GAE 的训练将这部分正确的拓扑信息凸显出来并学习到隐藏表征 Z 中。

至此, 以时刻 t 为例, GSAERU 通过图表征学习模块得到了 k 个表征矩阵 $\{Z_i^t | i = 1, \dots, k\}$, 且其规模均为 $c \times b$, 通过这些图 Sketch 上的表征即可求得时序图任意时刻 t 上节点 v_i 的表征 z_i^t :

$$z_i^t = \Gamma(z_{1,h_1(v_i)}^t, z_{2,h_2(v_i)}^t, \dots, z_{k,h_k(v_i)}^t) \quad (3-6)$$

其中 $z_{1,h_1(v_i)}^t$ 为第 1 个图 Sketch 上得到的表征矩阵的第 $h_1(v_i)$ 行向量, $\Gamma(\cdot)$ 为聚合函数, 这里我们采用取平均值作为聚合方法。

同时, 注意到 c 和 b 都是常数, 所以通过 GSAERU 模型的图表征模块学

习到的表征规模是固定的，不随时序图中节点的改变而变化。相较于其他工作 [41, 42] 要求图中节点数不能随时间变化，GSAERU 解决了“节点随时间推移而增减后其特征无法加入 RNN 训练”的问题。此外，由于输入 GAE 的图 Sketch 规模远远小于原图 ($c \ll n'$)，所以其训练时间与耗费的资源都远远少于大规模图数据训练。这两点优势体现了本工作在处理大规模图数据节点表征上的优越性。

3.3.3 时序学习模块

上一节介绍了如何生成每个时刻时序图上的节点表征。在获得了这些特征序列后，为了学习时序图在时间维度上的特征信息，GSAERU 模型引入了循环神经网络，这是因为循环神经网络能够挖掘出数据中的时序信息，对具有序列特性的数据非常有效。目前应用最广泛的循环神经网络模型有两个：长短期记忆网络 [51] (Long Short-Term Memory, LSTM) 与门控循环神经网络 [52] (Gated Recurrent Unit, GRU)，GSAERU 模型采用 GRU 模型来进行训练，这是因为 GRU 在效果上与 LSTM 接近但所需消耗的资源更少，训练更快 [53]。

GRU 的结构如图 3-6 所示。其输入是当前时刻的输入 x_t 和上一时刻得到的隐状态 h_{t-1} (隐状态包含了之前时刻的相关信息)，通过重置门控与更新门控计算得到当前时刻的输出 y_t 和传递给下一时刻的隐状态 h_t 。

取时序图中一段连续的时间序列 $\{\mathcal{G}^t, \mathcal{G}^{t+1}, \dots, \mathcal{G}^{t+T-1}\}$ ，下面将介绍如何通过这些快照预测时序图在时刻 $t+T$ 时的情况 $\hat{\mathcal{G}}^{t+T}$ 。

首先通过图表征学习模块计算得到时刻 t 时的隐藏表征集合 $\{Z'_i | i = 1, \dots, k\}$ ，首先将其拼接形成一个新矩阵 $M_t = (Z'_1 \| Z'_2 \| \dots \| Z'_k)$ ， $M_t \in R^{c \times kb}$ 。GSAERU 模型利用 M_t 作为 GRU 时刻 t 时的输入，其计算过程如下：

$$\begin{aligned}
 R_t &= \sigma(M_t W_{mr} + H_{t-1} W_{hr} + b_r) \\
 Z_t &= \sigma(M_t W_{mz} + H_{t-1} W_{hz} + b_z) \\
 \tilde{H}_t &= \tanh(M_t W_{mh} + (R_t \odot H_{t-1}) W_{hh} + b_h) \\
 H_t &= Z_t \odot H_{t-1} + (1 - Z_t) \odot \tilde{H}_t
 \end{aligned} \tag{3-7}$$

其中 H_{t-1} 为上一时刻的隐藏状态， b_r 和 b_z 为待学习参数， W_{mr} ， W_{hr} ， W_{mz} ， W_{hz} ， W_{mh} ， W_{hh} 为待学习的参数矩阵。本工作为了方便后续的预测，将隐藏状态 H_{t-1} 的规模设置为与输入 M_t 相同。

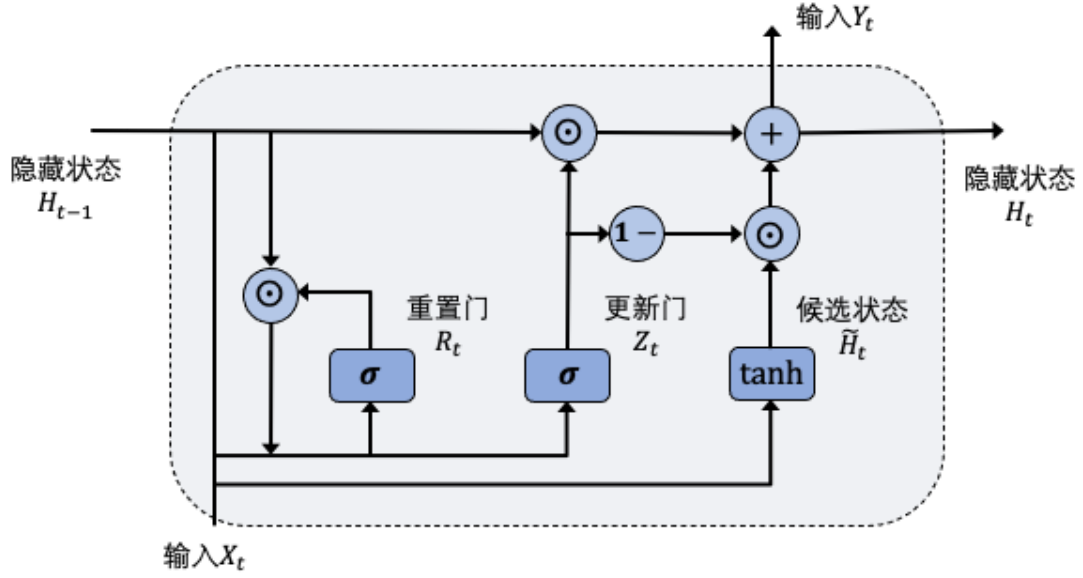


图 3-6: 门控循环神经网络

以此类推，最终得到时刻 $t + T - 1$ 时的输出 H_{t+T-1} 。由于规模相同，可以认为 H_{t+T-1} 就是对时序图 $t+T$ 时刻特征 $\hat{M}_{t+T}$ 的预测，即 $\hat{M}_{t+T} = H_{t+T-1}$ 。接下来将 $\hat{M}_{t+T}$ 重新拆分为 k 组特征矩阵 $\{\hat{Z}_i^{t+T} | i = 1, \dots, k\}$ ，对于任意的节点对 (v_i, v_j) ，计算它们在时刻 $t+T$ 时的特征，并将该节点对的特征进行内积计算，根据计算结果判断它们之间在时刻 $t+T$ 时是否存在边：

$$\begin{aligned}
 \hat{z}_i^{t+T} &= \Gamma(\hat{z}_{1,h_1(v_i)}^{t+T}, \hat{z}_{2,h_2(v_i)}^{t+T}, \dots, \hat{z}_{k,h_k(v_i)}^{t+T}) \\
 \hat{z}_j^{t+T} &= \Gamma(\hat{z}_{1,h_1(v_j)}^{t+T}, \hat{z}_{2,h_2(v_j)}^{t+T}, \dots, \hat{z}_{k,h_k(v_j)}^{t+T}) \\
 p_{ij}^{t+T} &= \sigma(\hat{z}_i^{t+T} (\hat{z}_j^{t+T})^T) \\
 \hat{a}_{ij}^{t+T} &= \begin{cases} 0 & p_{ij}^{t+T} < \alpha_{rec} \\ 1 & p_{ij}^{t+T} \geq \alpha_{rec} \end{cases}
 \end{aligned} \tag{3-8}$$

其中 $\hat{z}_i^{t+T}$ 和 $\hat{z}_j^{t+T}$ 分别为节点的预测表征， $\Gamma(\cdot)$ 为聚合函数（这里的方法为取平均值）， $\sigma(\cdot)$ 为 Sigmoid 函数， p_{ij}^{t+T} 为两个节点在 $t+T$ 时刻存在边的概率， α_{rec} 为实验设置的超参数，当预测概率超过该值时即可认为两节点之间存在一条边。通过计算 $\{\hat{a}_{ij}^{t+T} | 1 \leq i, j \leq n_{t+T}\}$ 即可预测时刻 $t+T$ 时时序图的邻接矩阵 $\hat{A}^{t+T}$ 。

在损失函数方面，图时序学习模块也选择交叉熵函数作为损失函数进行

GRU 模型的训练:

$$\mathcal{L}_{GRU} = -\frac{1}{n_{t+T}^2} \sum_{i=1}^{n_{t+T}} [\sum_{j=1}^{n_{t+T}} (a_{ij}^t \log \hat{a}_{ij}^t + (1 - a_{ij}^t) \log(1 - \hat{a}_{ij}^t))] \quad (3-9)$$

在完成训练后,即可通过获取时序图中某个时间段的一系列快照,将这些快照输入图表征学习模块得到隐藏表征,再将得到的表征输入图时序学习模块,预测下一时刻该时序图中的结构信息与获得节点的低维高质量表征。至此,GSAERU 模型的详细结构与执行流程全部介绍完毕。

3.4 实验分析

本节中,我们在四个真实世界的大规模图数据集对 GSAERU 模型进行了大量实验,并通过和现有动态图神经网络方法的对比展现其训练的快捷性和结果的准确性。

3.4.1 实验设置

3.4.1.1 实验环境

我们使用了基于 Python3.6.8 版本的深度学习库 PyTorch 实现了上述 GSAERU 模型,本实验在个人计算机上运行:

- CPU: Intel Xeon E5 2620 v2@2.10GHz;
- GPU: GeForce RTX 2070 8G;
- 内存: 64GB;
- 操作系统: 64-bit CentOS Linux 7.2。

3.4.1.2 数据集

我们使用四个真实世界中的大规模图数据集来评估模型,数据集的具体数据见表 3-1:

- **Pubmed**[54]: Pubmed^①是一个引文网络,包含来自 MEDLINE、生命科学期刊和在线书籍中超过 2600 万次的生物医学文献引用,引文可能包括指向 PubMed Central 和出版商网站的全文内容的链接。

^①<https://pubmed.ncbi.nlm.nih.gov/download/>

- **DBLP[55]**: DBLP^① 是从参考书目网站上提取的学术合著图。其中每个节点代表一个作者，每条边表示两位作者合作发表了一篇文章。
- **Tmall[55]**: Tmall^②是从天猫商城 2014 年的销售数据中提取的二分图，其中每个节点代表一个用户或一件商品，每条边代表一次购买，带有时间戳。五个最常购买的类别在实验中被视为标签。
- **Amazon[7]**: Amazon^③是一个从亚马逊官网获取的由用户互动和评价信息组成的大规模图数据集。

表 3-1: 数据集的统计数据

数据集名称	节点数	边数	特征维度	类别数
Pubmed	19,717	44,338	500	3
DBLP	28,085	236,894	600	10
Tmall	577,314	4,807,545	300	5
Amazon	1,598,960	132,169,734	200	107

3.4.1.3 基准算法

我们将 GSAERU 模型的性能与以下 7 个基准算法进行了比较。这些算法使用了静态图和时序图上的图表示学习方法。

- **DeepWalk[3]**: DeepWalk 采用随机游走的方法从静态图中采样生成有序的节点序列，以此为目标节点创建上下文，之后使用 skip-gram 模型从这些序列中学习节点的表示。
- **Node2Vec[56]**: Node2Vec 在 DeepWalk 的基础上通过带偏的随机游走来探索节点的邻域并学习静态图上的节点表示。
- **M²DNE[55]**: M²DNE 设计了一个时间注意点过程和一个通用动力学方程来捕获时序图的结构和时序特征。
- **DynamicTriad[23]**: DynamicTriad 基于图中的三元闭合子集对图结构信息和图演化模式进行建模，使模型能够有效地捕获图的时序信息。
- **MPNN-LSTM[57]**: MPNN-LSTM 是消息传递神经网络 (MPNN) 的时间序列

^①<https://paperswithcode.com/dataset/dblp>

^②<https://www.ceicdata.com/en/china/taobao-and-tmall-online-sales/cn-tmall-online-sales-value>

^③<https://data.world/datafiniti/consumer-reviews-of-amazon-products>

版本，具有两层 LSTM。MPNN 可以基于编码器动态捕获时序图中的长距离时间依赖性。

- **EvolveGCN[27]**: EvolveGCN 使用 RNN 在每个时间步更新 GNN 模型参数，有效地进行模型自适应学习并捕获时序图的时序信息。
- **TGAT[58]**: TGAT 从自我注意机制出发，设计出一个时间函数编码器以有效地学习图的时序和拓扑信息，从而为不同的动态图学习任务取得最先进的结果。

3.4.1.4 评估指标

本文使用平均精度均值（Mean Average Precision, MAP），曲线下面积值（Area Under Curve, AUC）和 F1 分数（F1-score）来衡量模型在链路预测任务上的性能。使用 Micro-F1 值和 Micro-Fi 值来衡量模型在节点多分类任务上的性能。

3.4.2 数值结果

本节中我们将使用具体的实验结果分析 GSAERU 模型的性能。为了处理动态图数据，我们首先将图数据集根据节点在时间维度上的信息划分为一个图快照序列。在此图序列中，我们以 5 为步长，有重叠地采集图子序列形成一个个样本，并以下一时刻的图快照作为当前样本的标签，这样就得了个完整的时序图训练样本。之后我们按照 70%，20% 和 10% 的比例将样本集合划分为训练集，测试集和验证集，并在此基础上进行模型训练。

3.4.2.1 资源使用和训练时间比较

在本节中，我们比较了在链路预测任务上各个模型训练时的资源使用情况和完成训练所需时间，如表 3-2 所示。由于 DeepWalk 与 Node2Vec 方法完全不考虑图中节点的特征信息，只使用了图结构信息进行训练，所以它们的内存消耗与训练时间都明显少于其他传统的动态图学习方法。而 GSAERU 模型由于引入了图摘要机制，其训练的子图规模远远小于原图，这就导致了其资源占用和训练时长都远远小于其他动态图表示学习算法。与 EvolveGCN 算法相比，GSAERU 算法在四个数据集上的平均内存消耗和平均训练时长分别只有其 9.86% 与 13.27%，这是因为 EvolveGCN 将 LSTM 中隐藏门的参数矩阵替换为

了图卷积参数矩阵，导致每次训练 LSTM 时需要将整张图输入，造成大量的内存占用。同时也反映出 GSAERU 算法在节省内存资源与加快训练速度上的优异表现。

表 3-2: 各模型训练时内存占用与训练时间比较

Method	Pubmed					DBLP				
	CPU	GPU	Epochs	Time per epoch(s)	Total Time (min)	CPU	GPU	Epochs	Time per epoch(s)	Total Time (min)
	Mem (MB)	Mem (MB)				Mem (MB)	Mem (MB)			
DeepWalk	15.51	0	120	0.09	0.18	77.81	0	160	0.37	0.99
Node2Vec	17.34	0	120	0.11	0.22	82.71	0	160	0.42	1.12
M ² DNE	42.35	34.05	150	0.74	1.85	237.21	45.41	177	2.59	7.64
DynamicTriad	41.02	20.88	130	0.62	1.34	127.33	64.47	180	1.74	5.22
MPNN-LSTM	53.37	45.08	138	0.73	1.68	275.88	54.45	175	2.54	7.41
EvolveGCN	93.00	55.73	150	0.79	1.98	312.63	58.54	168	3.02	8.46
TGAT	71.95	45.32	150	0.81	2.03	152.46	64.51	180	1.94	5.82
GSAERU	8.21	7.89	130	0.08	0.17	21.42	12.34	162	0.27	0.73

Method	Tmall					Amazon				
	CPU	GPU	Epochs	Time per epoch(s)	Total Time (min)	CPU	GPU	Epochs	Time per epoch(s)	Total Time (min)
	Mem (MB)	Mem (MB)				Mem (MB)	Mem (MB)			
DeepWalk	416.95	0	208	19.32	66.98	1734.23	0	250	27.95	116.46
Node2Vec	560.61	0	221	24.15	88.95	1965.14	0	250	30.47	126.96
M ² DNE	1233.86	124.33	250	35.88	149.50	5345.25	131.28	272	56.21	254.82
DynamicTriad	1283.26	98.83	234	30.81	120.16	6941.23	113.81	265	50.51	223.09
MPNN-LSTM	1463.87	74.34	253	37.41	157.75	6532.89	84.27	285	57.87	274.88
EvolveGCN	1808.65	129.70	274	42.78	195.36	6724.06	123.90	300	68.32	341.60
TGAT	1937.49	82.94	270	45.79	206.06	7672.49	115.83	305	80.32	408.29
GSAERU	253.31	36.82	220	7.59	27.83	354.85	47.33	275	9.61	44.05

3.4.2.2 链路预测任务

在本节中我们比较了各个图表示学习方法在链路预测任务上的性能，如表 3-3 所示。由于 DeepWalk 与 Node2Vec 方法只利用了图的拓扑信息，所以其在各个性能指标上的效果都远远低于其它方法。而对于 GSAERU 模型，虽然其没能在各个数据集上的各指标中均取得最优的性能，但 GSAERU 模型在性能上能够超越部分动态图表示学习算法（如 M^2DNE 与 $DynamicTriad$ ），与其它模型在大部分指标上的差距均不超过 1.5%，且与性能最好的算法在 F1 指标下的差距不超过 3.5%。这是因为使用图摘要对图进行“压缩”会不可避免地丢失

精度，而 GSAERU 模型采取的策略正是牺牲模型的精度来缩短训练时间和节省计算资源，与上一节中的实验结果对比，GSAERU 模型虽然牺牲了 1.5% 的模型精度，但平均缩短了 88.7% 的训练时间，节约了 89.3% 的计算资源。

表 3-3: 图表征学习算法在链路预测任务上的性能比较

Method	Pubmed			DBLP		
	MAP	AUC	F1	MAP	AUC	F1
DeepWalk	0.334	0.587	0.634	0.216	0.539	0.522
Node2Vec	0.346	0.562	0.513	0.367	0.546	0.497
M²DNE	0.414	0.675	0.682	0.474	0.547	0.487
DynamicTriad	0.552	0.736	0.716	0.544	0.689	0.748
MPNN-LSTM	0.570	0.712	0.843	0.590	0.742	0.747
EvolveGCN	0.578	0.826	0.869	0.592	0.781	0.775
TGAT	0.595	0.813	0.865	0.604	0.773	0.796
GSAERU	0.569	0.797	0.844	0.583	0.754	0.769

Method	Tmall			Amazon		
	MAP	AUC	F1	MAP	AUC	F1
DeepWalk	0.243	0.478	0.433	0.168	0.418	0.536
Node2Vec	0.338	0.512	0.410	0.232	0.453	0.523
M²DNE	0.334	0.610	0.533	0.207	0.439	0.502
DynamicTriad	0.374	0.699	0.543	0.259	0.603	0.613
MPNN-LSTM	0.420	0.784	0.695	0.357	0.664	0.690
EvolveGCN	0.442	0.803	0.709	0.413	0.651	0.626
TGAT	0.465	0.796	0.712	0.396	0.642	0.659
GSAERU	0.437	0.769	0.695	0.389	0.639	0.646

3.4.2.3 节点多分类任务

在本节中我们比较了各个图表示学习方法在链路预测任务上的性能，如表 3-4 所示。GSAERU 模型在各指标下的性能明显优于 *M²DNE* 与 *Dynamic-Triad* 方法，部分优于 MPNN-LSTM 算法，且 GSAERU 模型在 Micro-F1 的指标

下与最优效果模型相比平均降低了 2.73%，在 Macro-F1 的指标下与最优效果模型相比平均降低了 2.91%，其中最多降低了 3.73% 和 3.61%。

表 3-4: 图表征学习算法在节点多分类任务上的性能比较

Method	Pubmed		DBLP	
	Micro-F1	Macro-F1	Micro-F1	Macro-F1
DeepWalk	85.44	85.35	66.38	67.12
Node2Vec	87.33	88.95	67.31	66.93
M²DNE	89.87	88.26	69.71	69.75
DynamicTriad	90.34	89.49	66.95	66.42
MPNN-LSTM	90.05	90.31	67.74	65.05
EvolveGCN	91.37	90.47	71.32	71.20
TGAT	92.58	92.55	73.12	72.15
GSAERU	90.01	89.95	69.39	68.54

Method	Tmall		Amazon	
	Micro-F1	Macro-F1	Micro-F1	Macro-F1
DeepWalk	57.88	49.53	56.58	57.32
Node2Vec	60.66	54.58	53.24	53.10
M²DNE	64.65	58.47	60.43	60.77
DynamicTriad	60.72	51.16	61.82	62.09
MPNN-LSTM	58.07	50.27	63.03	62.96
EvolveGCN	61.77	55.78	64.67	65.41
TGAT	63.32	58.01	65.03	66.11
GSAERU	60.78	54.93	62.88	63.75

3.4.3 数据可视化

为了更直观地体现图表示学习得到的图中节点表征质量，本节使用 GCN，GAT，GraphSaint 和 GSAERU 模型在 Pubmed 数据集上通过单时间步的静态图进行 200 个轮次的训练。在得到图中节点的表征后通过一个有效的降维算法 TSNE[59] 将表征降维至二维空间，可视化结果如图 3-7 所示。从图中可以看出

通过 GSAERU 模型训练得到的表征和其它利用原图训练得到的表征在分类集群和相对位置上都非常相似。这预示了其能在节点分类任务中取得优异的效果，更表明在图摘要中训练得到的表征其效果近似于从原图上训练得到的表征。

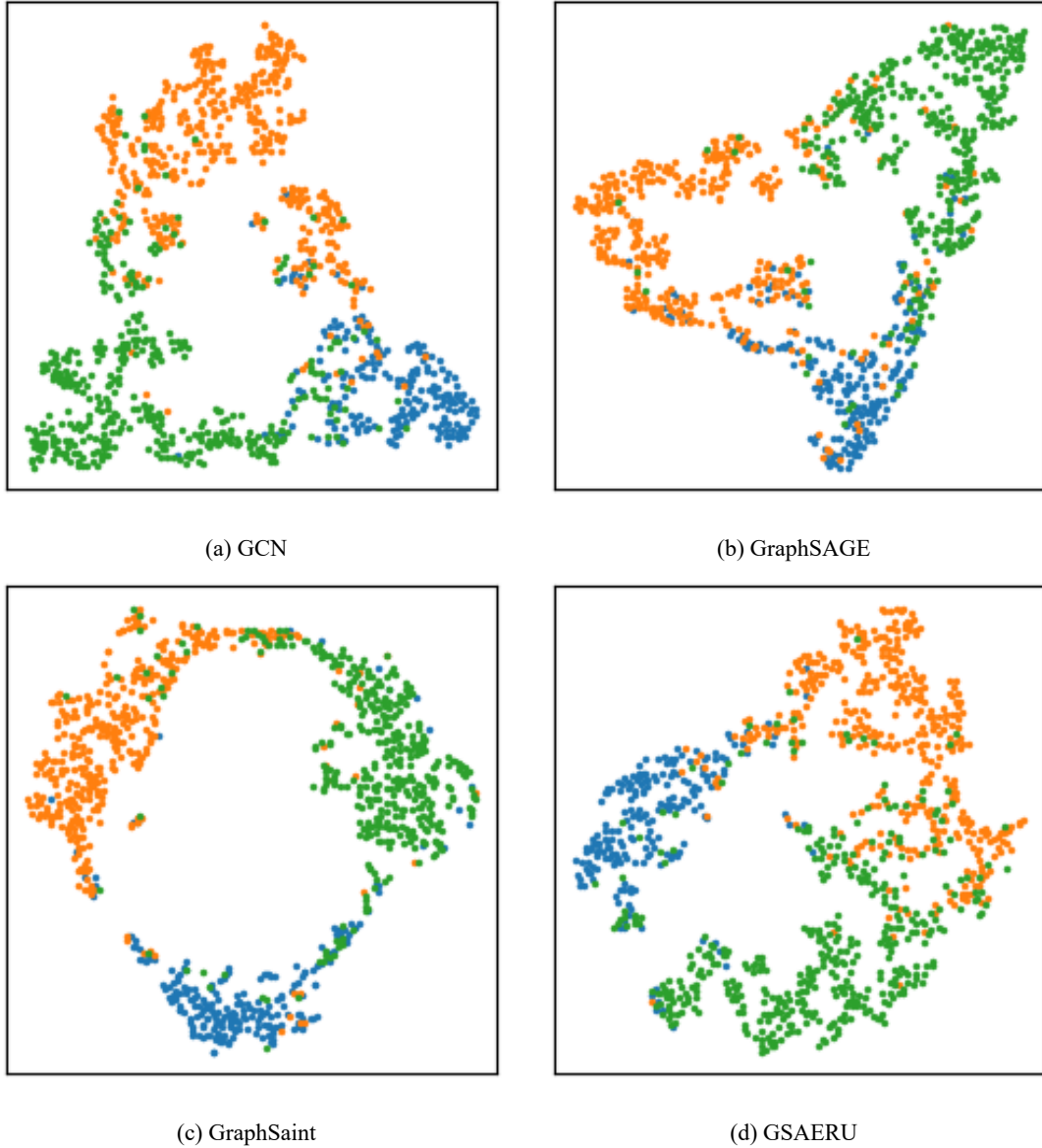


图 3-7: 聚类效果展示

3.4.4 超参分析

本节中我们对实验中的两个超参数：（1）生成图摘要的数量 k ；（2）图摘要的压缩率 r ($r = \frac{\epsilon}{n}$) 进行了分析，其结果如图 3-8 所示。图 3-8(a) 展示了随

随着生成图摘要数量的增多,模型在 F1 值上的性能也进行了相应的提升。这说明通过多个图摘要联合学习原图的正确拓扑结构这一做法是有效的,而当 k 的值超过 10 时 F1 的提升速度大大降低,说明 k 取值在 10 左右时就可以学习到绝大部分正确的原图拓扑信息。考虑到训练多个图自编码器也会给系统运行造成负担,最终我们将 k 的值定为 8。图 3-8(b)显示了压缩率对 F1 值的影响,当压缩率过低时模型无法学习到正确的原图拓扑信息,这是因为大量的节点映射到少量的节点中,导致生成的图摘要变成了一张全连接图,也就失去了学习图中拓扑信息的能力。而当压缩率提高到 5% 以上时,模型的精度随压缩率的提升而得到快速提升,当图摘要的节点规模达到原图 8% 的节点数时,其在 F1 上的性能就非常接近在原图上的结果了。

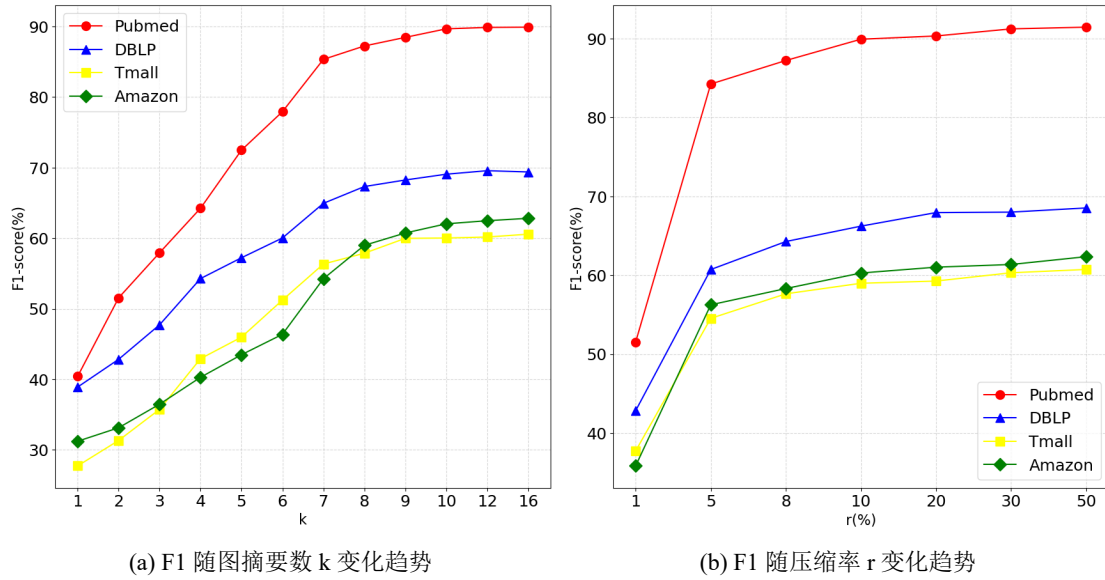


图 3-8: 超参分析

3.4.5 消融实验

为了研究 GSAERU 模型中不同模块的有效性,我们分别移除了部分模块并进行消融实验,其结果如表 3-5 所示,“ $\times$ ”表示在模型中移除了该模块。当模型使用所有模块功能时性能达到最佳,分别移除 SimHash 模块、图自编码器模块和时序学习模块后模型性能有了明显下降,而移除图摘要模块后整个模型的性能达到最低。这证明了使用图摘要的时序图表示学习模型 GSAERU 的优越性。

表 3-5: 消融实验：不同模型结构对 GSAERU 模型性能的影响分析

Datasets	SimHash	Graph Sketch	GAE	GRU	MAP	AUC	F1
Pubmed	×	×	✓	✓	0.225	0.438	0.494
	×	✓	✓	✓	0.477	0.722	0.819
	×	✓	✓	×	0.297	0.535	0.610
	✓	✓	✓	×	0.354	0.572	0.711
	✓	✓	×	✓	0.463	0.587	0.732
	✓	✓	✓	✓	0.569	0.797	0.844
DBLP	×	×	✓	✓	0.375	0.577	0.594
	×	✓	✓	✓	0.517	0.719	0.708
	×	✓	✓	×	0.405	0.592	0.583
	✓	✓	✓	×	0.497	0.667	0.697
	✓	✓	×	✓	0.490	0.685	0.658
	✓	✓	✓	✓	0.583	0.754	0.769
Tmall	×	×	✓	✓	0.294	0.519	0.495
	×	✓	✓	✓	0.419	0.721	0.634
	×	✓	✓	×	0.383	0.563	0.508
	✓	✓	✓	×	0.401	0.707	0.612
	✓	✓	×	✓	0.399	0.683	0.597
	✓	✓	✓	✓	0.437	0.769	0.695
Amazon	×	×	✓	✓	0.276	0.462	0.478
	×	✓	✓	✓	0.359	0.602	0.607
	×	✓	✓	×	0.285	0.473	0.596
	✓	✓	✓	×	0.338	0.597	0.603
	✓	✓	×	✓	0.291	0.495	0.529
	✓	✓	✓	✓	0.389	0.639	0.646

3.5 本章小结

本章我们提出了一种基于图摘要的大规模时序图表示学习方法。我们从时序图的“快照”模型出发，通过将时序图建模为一系列静态图的序列。并在序列中的每张静态图上使用一种基于图摘要的图表示学习方法快速得到当前时刻的图节点表征。最后将这些不同时刻的图节点表征组合形成时间序列特征送入循环神经网络中进行训练，达到捕获时序图的时序信息的目的。我们基于四个真实世界中的大规模图数据集，将本算法与六个当前热门的动态图学习方法进行对比。实验结果表明，在大规模图数据集中我们的模型能够有效减少计算资源并能够更快地完成模型训练。

第四章 基于图摘要的分布式图神经网络优化训练算法

4.1 研究动机

在深度学习领域，图神经网络（Graph Neural Networks, GNNs）被认为是处理图数据学习任务的有效方法，它可以将图结构信息融入到深度学习中，从而取得明显优于传统数据挖掘方法和机器学习的效果。如今 GNNs 已被应用于现实世界中的许多领域，如社交网络分析 [60, 61]，生物学中的蛋白质结构分析 [62]，购物网站商品推荐 [63–66] 和自然语言处理 [67, 68]。近年来，研究人员致力于 GNN 模型的设计，开发了包括 GCN[10]、GAT[15]、GraphSaint[7] 等一系列流行的图神经网络模型。同时，为了提高 GNN 模型开发效率，许多深度学习库提供了 GNN 开发的框架 [69–72]。另一方面，为了达到加速训练的目的，研究人员针对 GNN 训练的优化提出了一系列新方法，包括新的 GNN 运算内核（Computation Core）[73, 74]，用于加速图数据计算的编程模型 [75, 76]。然而，这些优化仅仅局限于在单个机器上训练 GNN，没有考虑到模型的可扩展性。

如今，真实世界中的网络图规模往往会非常巨大，以社交网络图（微博，微信）为例，其中包括上亿个顶点以及数十亿条边，而现有的大多数 GNN 模型无法处理大规模图数据集 [77] 或效率低下。因此，如何提高图神经网络模型在大规模图数据集上的训练性能对科研人员提出了新的挑战 [78, 79]。一种较为流行的解决方法是对原图使用采样 [7, 9, 80] 或蒸馏 [81–83] 技术得到一个规模远小于原图的“子图”，并以子图作为 GNN 模型的输入进行训练。这种做法虽然能大大提高训练速度，但模型性能最后会取决于所采集到的子图的质量，且由于子图会不可避免地丢失原图的结构信息，所以一定会导致模型预测准确度的降低。另一种方法是将分布式计算与 GNN 模型结合，因为在处理大规模图数据时，单个机器的内存和计算资源是有限的，往往会成为限制训练速度的瓶颈，而通过分布式计算能够调度更多的计算资源（如多 GPU 集群）来提高

训练效率。图4-1展示了分布式 GNN 训练的通用框架，其中包含了一个领导节点和若干工作者节点。对于输入的大规模图，首先由领导节点将图学习任务划分为一个个子任务，并将这些任务分配给工作者节点进行模型训练，工作者节点完成运算后将结果返回给领导节点，并由领导节点汇总结果并统一更新模型参数。

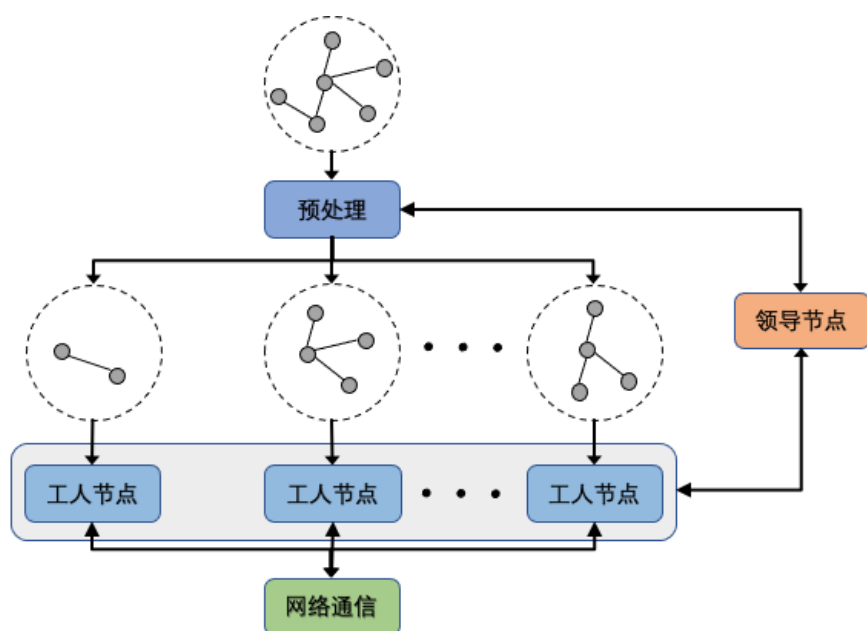


图 4-1: 分布式 GNN 训练通用框架图

更具体的，如图4-2所示，根据 [84] 提出的分类标准，我们将现有的分布式 GNN 框架分为四类，下面我们对其运行过程进行一一介绍。

- **分布式全批量训练中分派子图方法：**此类方法在每轮训练中对完整原图进行训练。首先将图数据和待训练的 GNN 模型输入领导节点，在训练过程中领导节点根据具体的节点特征聚合任务分发给各个工作者节点进行计算，然后收集返回结果并根据损失函数计算梯度，进行模型参数更新。其中的代表工作为 NeuGraph[28]，ROC[29]。然而，由于只有领导节点保存模型参数且训练时严重依赖于工作者节点的计算结果，导致这类方法的数据通信开销往往很大，且每轮训练的运行时间呈现不规律性。
- **分布式全批量预处理子图划分方法：**此类方法也是在每轮训练中使用对完整原图进行训练，在得到输入的图数据和 GNN 模型后，增加了预处理阶段使领导节点将图提前进行划分，并将每个划分后的子图和一份 GNN 模型的副本发送给对应的工作者节点，由它们在本地进行 GNN 模型的训练，最后

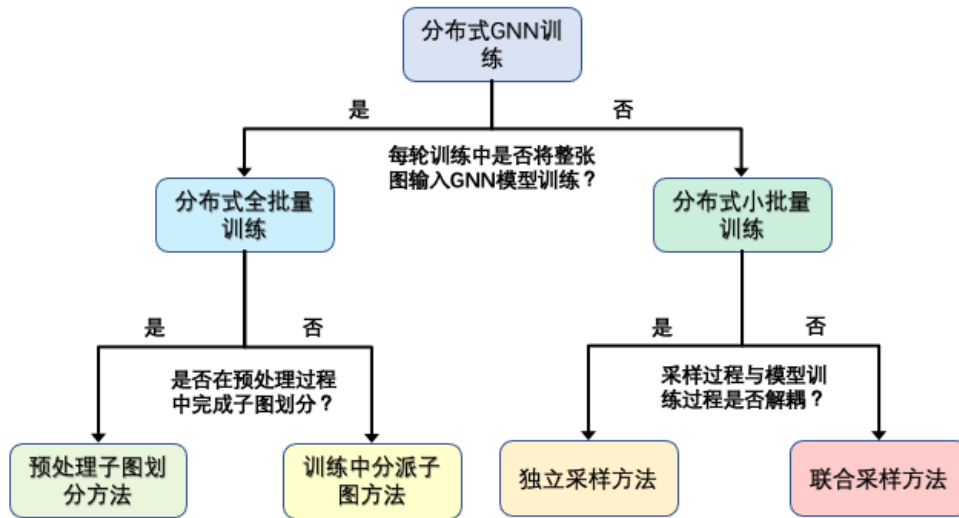


图 4-2: 分布式 GNN 框架的四种分类

将结果返回领导节点，由领导节点汇总后再统一通知工作者节点更新模型参数。其中的代表工作为 DistGNN[85]，FlexGraph[31] 和 SAR[86]。这类设计的好处是最大限度地调用了每个工作者节点的计算资源，缺点是在模型训练中工作者节点需要通过网络通信向其它节点获取其子图上待训练节点缺失的邻域信息，造成大量的通信开销。

- **分布式小批量独立采样方法：**此类方法首先由领导节点对输入图数据进行采样，并将采样得到的子图和输入 GNN 模型的副本分别发送给工作者节点，由工作者节点在本地根据子图进行 GNN 模型训练。梯度计算和模型参数更新仍是由领导节点来完成。其中的代表工作为 AGL[32]，SALIENT[33]。相较于前两类方法，本方法能够大大提升训练速度，但整体模型的运行效率仍由领导节点的采样效率来决定。
- **分布式小批量联合采样方法：**此类方法中领导节点不再对原图采样，而仅仅只是划分原图，并将划分子图和 GNN 模型副本发送给工作者节点。之后由工作者节点在得到的划分子图上进行采样并将采样结果作为本地 GNN 模型的输入进行模型训练。梯度计算和模型参数更新仍是由领导节点完成。其中的代表工作为 AliGraph[34]，2PGraph[35] 和 GraphTheta[87]。这类方法的性能瓶颈在于领导节点的原图划分效率和工作者节点上的采样训练效率。

针对上述分布式图神经网络模型的缺陷与不足，本章节提出了一种新的基于图摘要技术的分布式小批量图神经网络训练框架，称为 DSGNN，用于分布式场景下的图表征学习，该框架属于上述分类中的“分布式小批量联合采样方法”。对于输入的大规模图数据，首先由领导节点使用一个基于 Count-Min Sketch 的大图划分算法将原图划分为规模均匀的子图，并将子图和 GNN 模型的副本下发给每个工作者节点。工作者节点在本地执行一个基于 Sketch 技术的小批量图神经网络训练方法并将计算结果返回给领导节点。在领导节点收集到所有的返回结果后，其会利用这些结果进行梯度计算并统一更新所有工作者节点上的 GNN 模型参数。基于在真实世界的大规模图数据上的大量实验表明，DSGNN 能够有效地加速大规模图上的 GNN 模型训练，且其在各种性能指标上都优于目前最先进的算法。

4.2 问题建模

众所周知，真实世界的图数据往往是在不断变化的，为了体现这种动态性，本工作采用图流（Graph Stream）模型对真实世界中的动态图进行建模。

$$\begin{aligned}\mathcal{G} &= \langle e_1, e_2, \dots, e_m \rangle \\ e &= (x, y; t)\end{aligned}\tag{4-1}$$

如式 4-1 所示，图流本质上是一个序列。序列中元素 e 的含义是在时刻 t 出现了一条边 (x, y) （ x, y 为结点的编号）。显然该模型也能快速转化为传统图模型 $\mathcal{G} = (V, E)$ ，其中 V 是图中节点的集合， E 是边的集合且其规模为 m 。

本工作中，我们专注于节点级表征的学习，即将任意时刻 t 的图 $\mathcal{G}$ 转换到 c 维隐空间中，并尽可能地保存原来的结构信息与特征信息（ c 是一个常数且 $c \ll |V|$ ）。更通俗地说，模型的输入是一个由图流组成的序列，其中的节点具有一个 d 维的特征向量 $x_v \in R^d$ ，而模型输出的是图中每个节点 $v \in V$ 的 c 维表征向量 $z_v \in R^c$ （ $c \ll d$ ）。

4.3 模型设计和算法描述

在本节中提出了基于 DSGNN 框架进行分布式图神经网络训练的机制。我们首先提供了框架的整体流程，随后详细介绍了 DSGNN 框架的详细构成与训

练原理。

4.3.1 模型框架

端到端的分布式图神经网络训练框架 DSGNN 的总体结构如图 4-3 所示。该框架由一个领导节点和若干个工作者节点组成，其中领导节点主要负责数据预处理与模型参数同步管理，工作者节点主要负责具体的 GNN 模型训练。给定一个待训练的 GNN 模型与一组图流数据序列，首先由领导节点进行数据预处理，包括子图划分与 GNN 模型参数复制。随后领导节点将划分后的子图与 GNN 模型的副本分别分发给其它工作者节点。在每轮训练中，工作者节点根据获得的子图与 GNN 模型在本地进行小批量训练，训练完成后工作者节点将本地的损失函数值返回给领导节点。领导节点汇总所有工作节点的损失后将其聚合并计算全局梯度，依照全局梯度更新模型参数，并将更新结果发送给所有工作者节点进行同步模型更新。

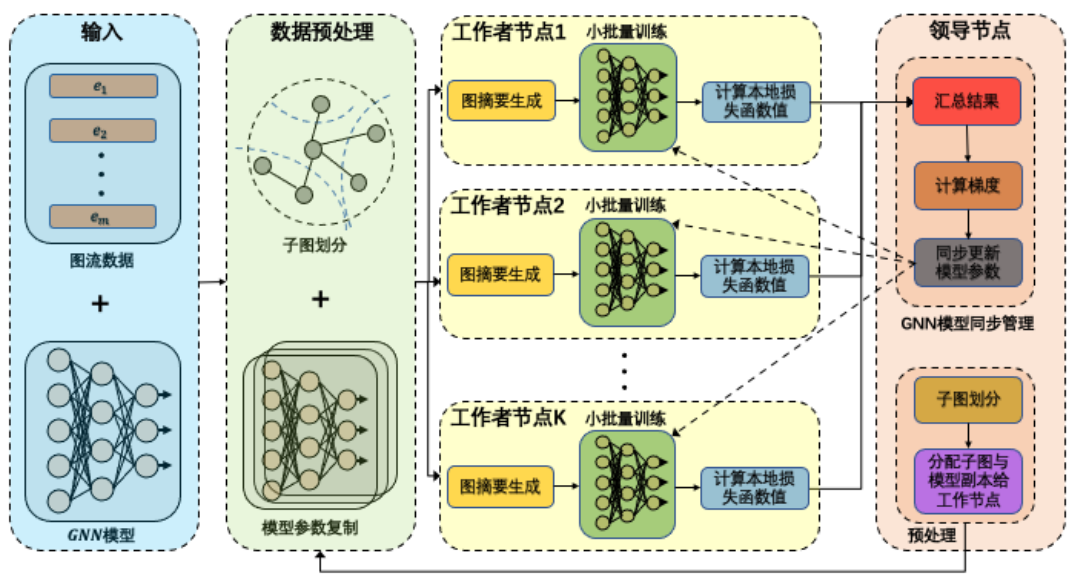


图 4-3: DSGNN 整体框架图

4.3.2 基于 Sketch-DBH 算法的子图划分

子图划分是将一个大规模图均匀地划分为一系列子图去适应分布式场景，划分后的子图被下发给各个工作者节点进行并行执行。如果当前子图需要其它

子图的信息，需要通过网络拓扑向其它节点发送查询请求，产生通信开销。因此子图划分的质量影响着每个节点的工作负载和工作节点之间的通信开销。目前主流的子图划分模式分为离线模式和流模式 [88]。离线模式是将完整图数据一次性载入内存后再进行划分，而流模式是按批次读取图数据并实时地将节点和边分配到其对应的子图中。对于大规模图数据而言，由于机器内存难以存储完整原图，故使用流模式分割更为适合。

因此，本工作在 DBH 分割算法 [89] (Degree Based Hashing) 的基础上引入 Count-Min Sketch 技术 (详见 2.2.3 节) 进行改进，形成了全新的流模式边分割子图划分算法——Sketch-DBH。下面介绍 DSGNN 框架中子图划分的具体过程。

4.3.2.1 初始化

首先，DSGNN 模型初始化一个 Count-Min Sketch 数据结构，其由 r 个相互独立的哈希函数 $h(\cdot)$ 组成，每个哈希桶的大小为 w 。将对于该 Count-Min Sketch 的查询抽象为一个函数 $CS(\cdot)$ ， $CS(v)$ 可以在 $O(r)$ 的时间内返回节点 v 在图中的度数。同时初始化一个规模为 l_{heap} 的最小堆 $minHeap$ 和一个邻接表 $N(\cdot)$ 用于维护每个节点的邻域信息 ($N(v)$ 会返回节点 v 的所有一阶邻居集合)。假设当前有 $\mathcal{K}$ 个工作者节点，那么需要将图划分为 $\mathcal{K}$ 份，因此额外初始化一个哈希函数 $o(\cdot)$ ，其哈希桶规模为 $\mathcal{K}$ 。记 $P(\cdot)$ 为划分子图的集合，每个划分子图 $P(index)$ 都可以通过其下标 $index$ 检索， $\bigcup_{i=1}^{\mathcal{K}} P(i)$ 等价于当前收集的所有图流数据。

4.3.2.2 图流数据处理

在大规模图流数据下，每当接收到一个三元组 $e = (u, v : t)$ 时，首先通过 Count-Min Sketch 查询两个节点 u 和 v 的度数 $CS(u)$ 和 $CS(v)$ ，并将其中的最小值输入到哈希函数 $o(\cdot)$ 中，即可得到该条边被分配到的工作者节点编号：

$$index = o(\min(CS(u), CS(v))) \quad (4-2)$$

接着对各数据结构中的信息进行更新。如图 4-4 所示，首先是 Count-Min Sketch，对于边 (u, v) ，分别计算它们在 r 个哈希表中的编号，即 $\{h_i(u) | i = 1, \dots, r\}$ 和 $\{h_i(v) | i = 1, \dots, r\}$ 并将其中对应下标的值加一。接着根据更新后

的 $CS(u)$ 和 $CS(v)$ 更新最小堆 $minHeap$ ，最小堆中的每个元素为一个键值对 $(x, CS(x))$ ，用于实时记录该流数据中度数最高的前 l_{heap} 个节点，若节点 u 和 v 已在最小堆中，则更新其值并更新整个最小堆，否则判断能否将其插入最小堆中。最后进行邻域表 N 更新，只需将节点 u 和 v 分别加入互相的邻域集合即可。

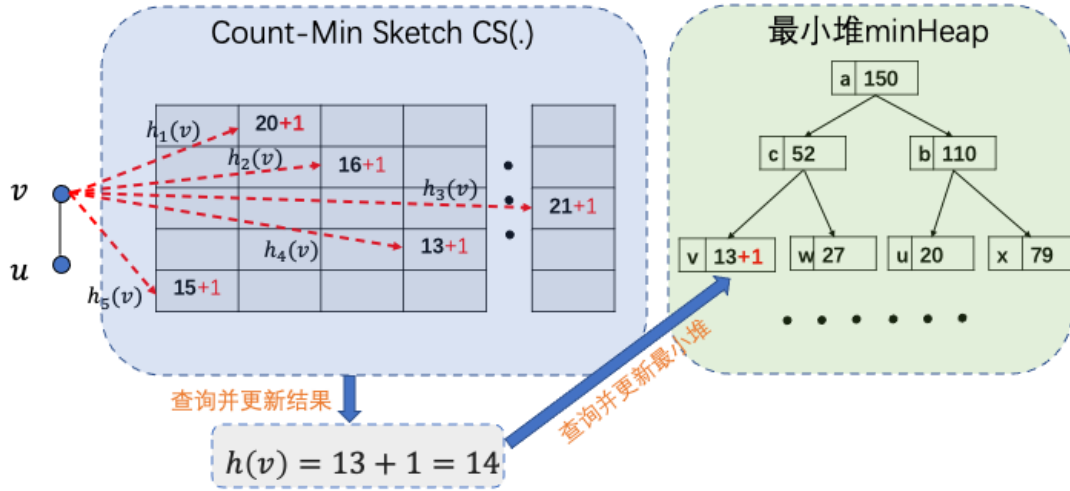


图 4-4: 图流数据处理与更新存储结果示例

4.3.2.3 分配子图

在上一节完成子图划分后得到了子图集合 $\{P(i)|i = 1, \dots, \mathcal{K}\}$ ，接下来需要将每个子图及其中节点的特征发送给对应的工作者节点，针对这一过程 DSGNN 模型进行了两项优化。

第一，对子图进行“扩展”。具体做法是对子图 $P(i)$ 中的每一个节点 v ，查询最小堆 $minHeap$ 中的所有键值对判断是否存在该键。若存在则将节点 v 的一阶邻居 $N(v)$ 全部加入子图 $P(i)$ 中。进行这一优化的原因是图中存在度数很大的节点，而 Sketch-DBH 算法是根据度小的节点计算子图的编号，这会导致这些高度节点在子图划分后大量边的缺失。以至于在后续的 GNN 模型训练中，为了聚合其缺失的邻域信息而向其它工作者节点发送大量的查询请求，严重拖慢模型的训练速度。而 Sketch-DBH 算法中维护一个最小堆的目的就是实时保存图流数据中的高频节点。

第二，对子图的包含的节点特征向量进行“压缩”。由于图中节点的初始特征向量维度往往较大（ d 的大小为数百至数千不等），在大规模图数据场景下这些节点特征向量的存储和传输也会消耗大量的资源。考虑到这些节点的特征向量中存在向量相同或相似的现象，DSGNN 模型引入了一个唯一性列表（unique list） $I_{feature}(\cdot)$ 用于存储所有不同的向量。针对每个子图 $P(i)$ ，首先将其中的每个节点 v 的特征向量 x_v 加入该子图对应的 $I_{feature}$ 中，并记录其下标 $index$ 。处理完成后只需将列表 $I_{feature}$ 和新的顶点集合 $\{(v, index)\}$ 发送给工作者节点即可。后续在工作者节点上通过查询 $I_{feature}(index)$ 即可得到节点 v 对应的特征向量 x_v 。这使得每个子图的特征向量存储成本从 $O(|P(i)|d)$ 降为了 $O(|I_{feature}|d)$ 。

4.3.2.4 时间复杂度分析

DSGNN 中的子图划分模块整体工作流程如算法 4.1 所示。得益于哈希表和 Count-Min Sketch 结构的引入，许多查询和更新操作都可以在常数时间内完成。每处理一条图流数据时，除了查询并更新 Count-Min Sketch 和最小堆的时间分别为 $O(r)$ 和 $O(l_{heap} \log_2 l_{heap})$ ，其余操作均可于 $O(1)$ 时间内完成，故处理每条图流数据的时间复杂度为 $O(r + l_{heap} \log_2 l_{heap})$ 。整个子图划分算法的时间复杂度为 $O(m(r + l_{heap} \log_2 l_{heap}) + |E| + \mathbb{E}_{v \in minHeap} \mathcal{K} L_N l_{heap})$ 。其中 $\mathbb{E}_{v \in minHeap}$ 为最小堆中的节点出现在每个子图中的期望， L_N 为邻接表 $N(\cdot)$ 中邻域节点的平均大小。

相较于其他子图划分算法，Sketch-DBH 算法能够高效计算每条图流数据应当被分配的子图编号，并实时维护一个记录图流数据中高频节点的最小堆。同时，该算法针对后续 GNN 训练时节点缺失邻域和传输节点特征开销过高这两个问题都提出了优化方法，解决了子图划分后高频节点大量边的缺失以及边的负载不均匀问题。

4.3.3 基于图摘要技术的小批量训练

经过上一节的子图划分，我们获得了 $\mathcal{K}$ 个子图，将其记为 $\{\mathcal{G}^1, \mathcal{G}^2, \dots, \mathcal{G}^{\mathcal{K}}\}$ 。DSGNN 框架会为每一个工作者节点 $worker[i]$ 分配一个子图 $\mathcal{G}^i$ 以及一份待训练的 GNN 模型 $GNN(\cdot)$ 的副本，之后由工作者节点在该子图上进行 GNN 模型的训练。然而在大规模图数据场景中，划分后的子图规模依然十分庞大，直接训练的成本很高。解决这一问题的常用方法是使用原图的小批量（Mini-Batch）样本代替原图输入到 GNN 模型中进行训练。DSGNN 框架从 GNN 模型的特征

算法 4.1 基于 Sketch-DBH 的子图划分算法

输入: 图流数据序列: $\{e_1, e_2, \dots, e_m\}$, $e = (u, v : t)$; 需要划分的子图个数: $\mathcal{K}$;

输出: 划分后的子图集合 $\{P(i) | i = 1, \dots, \mathcal{K}\}$

- 1: 初始化: $\mathcal{K}$ 个空集 $P(i) = \phi$, $i = 1, \dots, \mathcal{K}$; 一个空 Count-Min Sketch: $CS(\cdot)$; 一个长度为 l_{heap} 的空最小堆: $minHeap$; 一个空邻域表: $N(\cdot)$; $\mathcal{K}$ 个唯一性列表: $I_{feature,i}$, $i = 1, \dots, \mathcal{K}$; 一个容量为 $\mathcal{K}$ 的哈希函数: $o(\cdot)$
- 2: **for each** $e = (u, v : t) \in \{e_1, e_2, \dots, e_m\}$ **do**
- 3: $\delta_1 = degree(u) = CS(u)$
- 4: $\delta_2 = degree(v) = CS(v)$
- 5: $index = o(\min(\delta_1, \delta_2))$
- 6: $P(index) = P(index) \cup (u, v)$
- 7: $N(u) = N(u) \cup v$
- 8: $N(v) = N(v) \cup u$
- 9: $I_{feature,index} = I_{feature,index} \cup x_u \cup x_v$
- 10: 更新 Count-Min Sketch 和最小堆 $minHeap$
- 11: **end for**
- 12: **for each** $P(i) \in \{P\}$ **do**
- 13: **for each** $w \in P(i)$ **do**
- 14: **if** $w \in minHeap$ **then**
- 15: 将 $N(w)$ 中所有节点与 w 组成边并加入 $P(i)$ 中
- 16: **end if**
- 17: **end for**
- 18: **end for**
- 19: **return** $\{P(i) | i = 1, \dots, \mathcal{K}\}$

更新原理出发, 提出了基于图摘要技术生成原图小批量样本的方法, 在确保模型精度不过分降低的前提下大大减少了训练成本, 加速了训练过程。

GNN 模型的通用训练过程如算法 4.2 所示。从节点角度出发, 每一轮训练节点都会聚合其邻域中节点的嵌入并和自身的嵌入融合, 达到更新自身嵌入的目的。整个过程中节点的嵌入递归的依赖于其邻居节点的嵌入, 随着模型隐藏层的增加这样的更新操作会呈指数级增长, 从而出现”邻居爆炸“现象 [90], 这一现象会增加模型训练时的内存开销, 严重拖慢模型的训练速度。以图 4-5(a)为例, 对于节点 8, 其嵌入的更新依赖于二阶邻居 2、5 与三阶邻居 1、3、4、6、7。假设存在一个理想的哈希函数 $h(\cdot)$, 如图 4-5(b)所示, 其能将二阶邻居节点映射到一个新的节点 **a**, 将一阶邻居映射到一个新的节点 **b**, 在这样一张新图上对节点 8 进行嵌入更新, 其仍能学习到正确的邻域节点的信息, 但在新图上的训练开销会远小于原图。

基于这样的思想, DSGNN 框架在小批量训练中采用图摘要技术来生成子

算法 4.2 GNN 模型通用算法框架

输入：图： $\mathcal{G}$ ；每个节点的初始表征： $x_v \in R^d$ ；节点最远邻居的跳数： $k_{max} \in N$

输出：每个节点的嵌入向量： h_v

```

1:  $h_v^{(0)} = x_v$ 
2: for  $k = 1$  to  $k_{max}$  do
3:   for each  $v \in V$  do
4:      $S_v = \text{SAMPLE}(N(v))$ 
5:      $h'_v = \text{AGGREGATE}(h_u^{k-1}, u \in S_v)$ 
6:      $h_v^k = \text{COMBINE}(h_v^{k-1}, h'_v)$ 
7:   end for
8: end for
9:  $h_v = h_v^{k_{max}}$ 
10: return 图中所有节点  $v \in V$  的嵌入  $h_v$ 

```

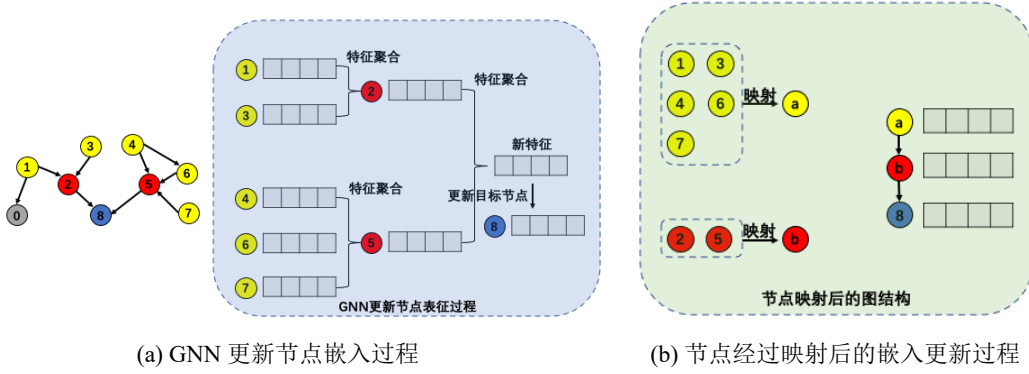


图 4-5: 图摘要示例

图。基于 sketch 的图摘要技术本质上也是通过一个哈希函数将所有原图节点映射到一个规模远远小于原图的新图上，由于这样的压缩必然会导致原图一部分拓扑信息的丢失，故图摘要技术采用生成多个图 Sketch 并通过在其上进行联合查询来纠正每个 Sketch 上的结构误差，这也正契合了小批量训练需要采样生成多个子图的做法。

以工作节点 $worker[i]$ 为例，在其接收到领导节点分配的子图 $\mathcal{G}^i$ 和待训练的 GNN 模型 $GNN(\cdot)$ 后，首先依照 3.3.2.1 节提出的图 Sketch 生成方法，根据 k 个两两相互独立的 *SimHash* 函数生成 k 个子图 $\{\mathcal{GS}_1^i, \mathcal{GS}_2^i, \dots, \mathcal{GS}_k^i\}$ ，并以这些子图作为小批量训练样本输入待训练的 GNN 模型中进行训练，并根据具体的模型以及任务计算损失函数值，得到损失函数集合 $\{l_1^i, l_2^i, \dots, l_k^i\}$ ，接下来需要通过聚合这些损失函数值得到该工作节点上的整体损失函数值 $l_{worker[i]}$ 并返回给领导节点。

$$l_{worker[i]} = \Gamma(l_1^i, l_2^i, \dots, l_k^i) \quad (4-3)$$

由于使用图 Sketch 技术生成的子图会无法避免地损失一部分原图的拓扑信息，故需要对每个 Sketch 在 GNN 模型上的训练结果进行纠偏。。在 3.3.2.2 节中，我们使用图自编码器将每个图 Sketch 还原成原图来使编码器学习图 Sketch 中包含的原图正确的拓扑信息，由此训练得到多个隐藏表征，并通过对这些表征取平均来得到原图节点的特征向量。而在 DSGNN 框架中，图 Sketch 要输入的 GNN 模型是不确定的，损失函数的计算也是不确定的，这就导致了无法再使用图自编码器对学习结果进行纠偏。

受到图注意力机制 [15] 的启发，DSGNN 框架采用一个基于损失函数值的注意力机制学习每个图 Sketch 中包含的原图正确拓扑信息的权重，并以此权重聚合多个 Sketch 上训练的结果来弥补各自缺失的原图拓扑，使得最终的训练效果接近在原图上训练的效果。其原理如图 4-6 所示。

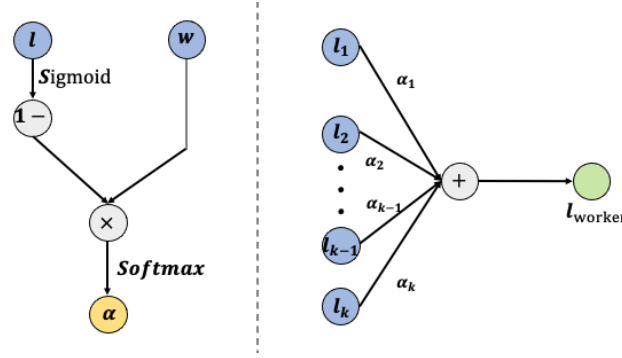


图 4-6: 基于图 Sketch 上损失函数值的注意力机制原理

在每轮训练中，首先将 k 个图 Sketch 输入待训练 GNN 模型并求得各自的损失函数值 $\{l_1^i, l_2^i, \dots, l_k^i\}$ ，显然若一个图 Sketch 训练得到的损失函数值越低，说明其包含越多的原图拓扑信息，则应该在损失函数值聚合阶段给予更高的权重，其权重以及该工作者节点上的整体损失函数值计算过程如式 4-4 所示。

$$\begin{aligned}
 l_j^i &= f_{\text{Loss}}(\mathcal{GS}_j^i) \\
 \delta_j &= \text{LeakyReLU}\left(\frac{w_j \exp(-l_j^i)}{1 + \exp(-l_j^i)}\right) \\
 \alpha_j &= \frac{\exp(\delta_j)}{\sum_{m=1}^k \exp(\delta_m)} \\
 l_{\text{worker}[i]} &= \sum_{j=1}^k (\alpha_j l_j^i)
 \end{aligned} \tag{4-4}$$

其中 $f_{\text{Loss}}(\cdot)$ 为具体模型的损失函数， $\vec{w}$ 为待学习的参数向量。至此，工

作者节点 $worker[i]$ 完成了一轮本地训练并得到了本地损失函数值 $l_{worker[i]}$ 。

4.3.4 全局梯度计算与模型参数更新

在 DSGNN 框架每轮训练中，领导节点依据来自 $\mathcal{K}$ 个不同的工作者节点返回的损失函数值来更新模型参数。首先计算全局损失函数 l_{global} 。

$$\begin{aligned} l_{global} &= \Gamma(l_1, l_2, \dots, l_K) \\ &= \frac{1}{K}(l_1 + l_2 + \dots + l_K) + \lambda R(w) \end{aligned} \quad (4-5)$$

其中 $R(w)$ 为正则项， λ 为正则系数。

接着领导节点利用全局损失函数值计算梯度并对本地 GNN 模型参数进行更新，之后将更新后的参数连同全局损失 l_{global} 发送给每个工作者节点。最后由工作者节点根据领导节点返回的新模型参数更新本地 GNN 模型，这样就确保了分布式图神经网络训练所要求的模型同步。同时利用 l_{global} 更新本地的损失函数注意力层的参数。

4.4 实验分析

本节中，我们在四个真实世界的大规模图数据集上对 DSGNN 框架进行了大量实验，并通过和现有的分布式图神经网络训练框架进行对比体现了本框架在加速训练上的有效性。

4.4.1 实验设置

4.4.1.1 实验环境

我们使用了基于 Python3.6.8 的 PyTorch 与 PyG 实现了上述 DSGNN 框架（PyG[71] 是 PyTorch 的几何深度学习扩展库，可以对图等不规则结构数据进行深度学习。同时支持 CPU 和 GPU 计算）。本实验在服务器上进行：

- CPU：Intel Xeon E5-2620 v2@2.10GHz CPU；
- GPU：4×GeForce RTX 2070 8G；
- 内存：64GB；
- 操作系统：64-bit CentOS Linux 7.2，

4.4.1.2 数据集

本节中，我们使用四个真实世界中的大规模图数据集来评估模型，数据集的具体统计数据见表 4-1。

- **Pubmed**[54]: Pubmed^①是一个引文网络，包含来自 MEDLINE、生命科学期刊和在线书籍中超过 2600 万次的生物医学文献引用，引文可能包括指向 PubMed Central 和出版商网站的全文内容的链接。
- **PPI**[19]: PPI^②是一个蛋白质相互作用图数据集，每个图代表一个人体组织，每个节点代表一种蛋白质，边则代表蛋白质之间的关系。
- **Reddit**[19]: Reddit^③是一个在线讨论论坛的社交网络数据集，每个节点代表用户发的一个帖子，每条边是帖子之间的评论。
- **OGBN-PRO**[77]: OGBN-PRO^④是由 Open Graph Benchmark 机构在 iMaterialist-2019 数据集基础上整理得到的一个在线产品数据集，每个节点代表一个产品，每条边代表两个产品之间的关系。

表 4-1: 数据集的统计数据

数据集名称	节点数	边数	特征维度	类别数
Pubmed	19,717	44,338	500	3
PPI	56,944	1,612,348	700	121
Reddit	232,965	114,848,857	602	41
OGBN-PRO	2,449,029	61,859,140	100	24

4.4.1.3 基准算法

我们首先将 DSGNN 框架应用于三个最流行的图神经网络——GCN[10]、GAT[15] 和 GraphSaint[7] 上，形成其分布式训练版本并与单机训练版本进行性能比较。接着将 DSGNN 框架与目前最流行的 3 个分布式图神经网络训练框架——ROC[29]，DistDGL[91] 和 Aligraph[34] 进行了对比。

^①<https://pubmed.ncbi.nlm.nih.gov/download/>

^②<https://paperswithcode.com/dataset/ppi>

^③<https://paperswithcode.com/dataset/reddit>

^④<https://paperswithcode.com/dataset/dyml-product>

4.4.1.4 评估指标

我们使用以下三个指标来评估算法在节点分类任务上的性能：

- 精确率（Precision, P）：正确判断为真的样本数占总的判断为真的样本数的比值。
- 召回率（Recall, R）：正确判断为真的样本数占总的真实真样本数的比值。
- F1 分数（F1 Score）：精确率与召回率的调和平均数，用于综合衡量一个分类模型的精确度。

其具体计算过程如下：

$$\begin{aligned} P &= \frac{TP}{TP + FP} \\ R &= \frac{TP}{TP + FN} \\ F1 &= 2 \times \frac{P \times R}{P + R} \end{aligned} \quad (4-6)$$

4.4.2 数值结果

4.4.2.1 资源使用和训练时间比较

本节中我们选取了 3 个经典的单机图神经网络模型——GCN、GAT 和 GraphSaint，并使用 DSGNN 框架实现了其分布式训练版本，接着比较了集中式与分布式两种训练方法下的内存使用情况与训练速度，其结果如表 4-2 所示，其中分布式环境下的 CPU 和 GPU 消耗所有机器消耗之和。从表中可以看出使用 DSGNN 框架后的 GNN 模型无论是内存占用还是训练速度都远远小于集中式训练的方法，这也体现了 DSGNN 框架在分布式图神经网络训练上的强大性能。

4.4.2.2 链路预测任务

在本节中，针对链路预测任务，我们首先比较了三种主流的 GNN 模型在集中式训练和基于 DSGNN 框架的分布式训练下的性能差异。接着将 DSGNN 框架与其它三种流行的分布式 GNN 训练框架进行对比，对比方法是在四个数据集上使用四种分布式框架分别训练最基本的 GCN 模型并比较其 F1 分数。实

表 4-2: GNN 模型与其分布式版本训练时内存占用与训练时间比较

Method	Pubmed					PPI				
	CPU Mem (MB)	GPU Mem (MB)	Epochs	Time per epoch(s)	Total Time (min)	CPU Mem (MB)	GPU Mem (MB)	Epochs	Time per epoch(s)	Total Time (min)
GCN	21.59	30.42	121	0.76	1.53	47.91	35.23	140	1.87	4.36
GCN+DSGNN	2.14	13.26	120	0.04	0.08	5.71	19.25	138	0.19	0.44
GAT	24.75	24.57	138	0.52	1.20	73.27	48.21	150	1.37	3.43
GAT+DSGNN	4.09	10.58	130	0.07	0.15	7.23	24.29	150	0.23	0.58
GraphSaint	41.17	31.96	140	0.45	1.05	172.78	44.29	180	1.74	5.22
graphSain+DSGNN	4.30	15.37	135	0.09	0.20	23.47	24.69	170	0.26	0.74

Method	Reddit					OGBN-PRO				
	CPU Mem (MB)	GPU Mem (MB)	Epochs	Time per epoch(s)	Total Time (min)	CPU Mem (MB)	GPU Mem (MB)	Epochs	Time per epoch(s)	Total Time (min)
GCN	759.97	37.34	160	4.95	13.2	2432.17	51.32	220	22.31	81.80
GCN+DSGNN	76.24	13.49	145	0.32	0.77	224.51	39.34	180	2.79	8.37
GAT	937.78	34.13	200	4.42	14.73	3579.19	65.97	240	28.69	114.76
GAT+DSGNN	81.09	12.48	180	0.63	1.89	324.39	54.63	235	3.69	14.45
GraphSaint	1418.39	42.51	185	5.13	15.82	4771.37	183.40	220	51.02	187.07
graphSain+DSGNN	102.13	7.63	160	0.75	2.00	329.74	86.34	200	5.49	18.30

验结果分别见表 4-3 和表 4-4。

在表 4-3 中, 采用 DSGNN 框架的分布式 GNN 模型在精确率 (P) 这一指标上的结果均低于集中式, 但在召回率 (R) 这一指标上的结果均高于集中式。这是由于原图通过图摘要技术形成超图后, 原先不相连的一对节点映射到的一对超点之间可能存在超边, 这一现象会误导 GNN 模型认为这两点之间存在联系, 从而学习到这一错误的拓扑信息, 导致在测试集上进行链路预测时产生更多的假正例结果, 进而使精确率降低, 召回率升高。另一方面, 所有的分布式 GNN 模型在 F1 指标上的结果都略微低于集中式模型, 但除了在 Reddit 数据集上两种版本的 GraphSaint 模型的 F1 值相差 1.9% 外, 其余数据集和模型上两者 F1 分数的差距均不超过 1%。这是由于在模型训练阶段引入了基于图摘要的小批量训练方法, 而经过图摘要生成的子图会不可避免地丢失原图的拓扑信息, 但相较于其缩短的训练时间和节约的计算资源, 我们认为这些微小的性能损失是可以接受的。

在表 4-4 中, 采用 DSGNN 框架实现的分布式 GCN 模型在四个数据集上的 F1 值均高于其它三种框架。由于 ROC 与 DistDGL 采用全批量训练方法, 这使

得其在面对大规模图流数据时会产生大量的通信开销以及同步开销，而这两种框架内置的通信队列在请求数量超出阈值时会触发丢弃机制，导致模型精度的降低。对于同样使用联合采样小批量训练方式的 Aligraph 框架，由于 DSGNN 使用了基于图摘要的小批量训练与基于损失函数的注意力机制，所以其性能较前者有了显著的提升。

表 4-3: 图表征学习算法在链路预测任务上的性能比较

Method	Pubmed			PPI		
	P	R	F1	P	R	F1
GCN	0.934	0.877	0.905	0.877	0.842	0.859
GCN+DSGNN	0.901	0.883	0.892	0.859	0.850	0.854
GAT	0.942	0.897	0.919	0.896	0.878	0.887
GAT+DSGNN	0.913	0.904	0.908	0.883	0.874	0.878
GraphSaint	0.947	0.918	0.932	0.917	0.902	0.909
GraphSaint+DSGNN	0.924	0.914	0.919	0.893	0.905	0.899

Method	Reddit			OGBN-PRO		
	P	R	F1	P	R	F1
GCN	0.825	0.794	0.809	0.743	0.775	0.759
GCN+DSGNN	0.799	0.803	0.801	0.729	0.783	0.755
GAT	0.854	0.817	0.835	0.761	0.765	0.763
GAT+DSGNN	0.820	0.833	0.826	0.735	0.782	0.758
GraphSaint	0.885	0.844	0.864	0.774	0.804	0.789
GraphSaint+DSGNN	0.836	0.851	0.843	0.745	0.793	0.768

表 4-4: 不同分布式 GNN 框架运行 GCN 模型的 F1 值 (%) 对比

Method	Pubmed	PPI	Reddit	OGBN-PRO
ROC	88.43	84.83	79.72	74.87
DistDGL	88.28	84.75	79.19	75.27
Aligraph	88.79	85.14	79.58	75.16
DSGNN	89.23	85.36	80.11	75.49

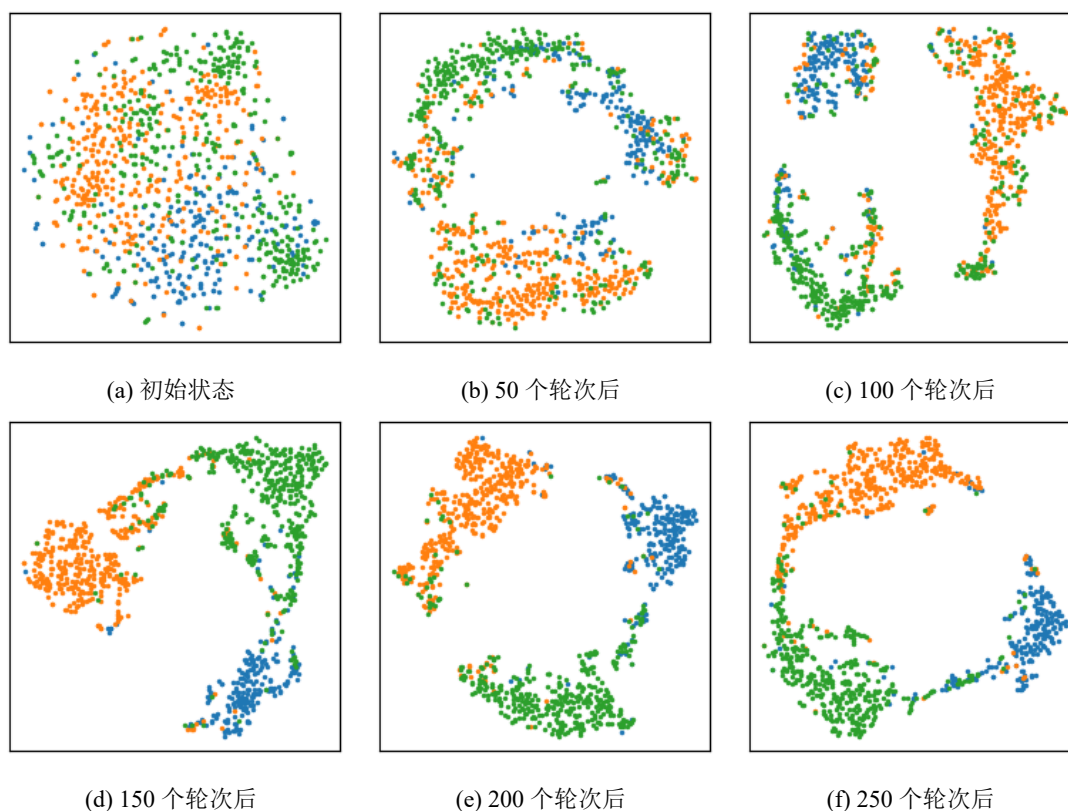


图 4-7: DSGNN+GraphSaint 训练过程可视化结果

4.4.3 数据可视化

在本节中我们使用 DSGNN 框架实现了 GraphSaint 模型的分布式训练版本并在 Pubmed 数据集上进行训练。每训练 50 个轮次后将模型输出的表征通过 TSNE 算法降维并可视化，结果如图4-7所示。可以看出原始数据中的各类节点经降维后呈现无序排列的状态，但经过多轮训练迭代后逐渐呈现分类聚集现象，并在 200 个训练轮次后形成边界分明的稳定聚类。这说明 DSGNN 框架下的分布式 GNN 模型训练不仅可以快速收敛，而且训练得到的表征能够对样本进行分类，体现了 DSGNN 模型的高效性。

4.4.4 超参分析

本节中我们对实验中的三个超参数：（1）生成图摘要的数量 k ；（2）图摘要的压缩率 r ($r = \frac{c}{n}$)；（3）工作者节点的数量 $\mathcal{K}$ 进行了分析，其结果如图4-8所示。图3-8(a)显示了随着 k 的增大，模型在 F1 值上的性能也进行了显著的提升。但当 k 的值大于 7 后，F1 指标的增长速度大大降低，表明此时通过

联合查询生成的 k 个图摘要已经可以还原原图正确的图拓扑信息。图 3-8(b) 显示随着压缩率的提高, F1 值也会相应增长, 这是因为随着图摘要规模的增大, 其能包含更多原图的拓扑信息, 从而提高模型性能, 在压缩率在 8% 附近时在图摘要上训练的效果就已经非常接近于在原图上训练的效果了。图 4-8(c) 显示了通过增加计算节点, 模型的训练时间呈接近线性的下降趋势。这也从侧面反映了我们的子图划分算法可以将大规模图均匀地分配到各个计算节点中, 不会出现负载不均衡的现象。

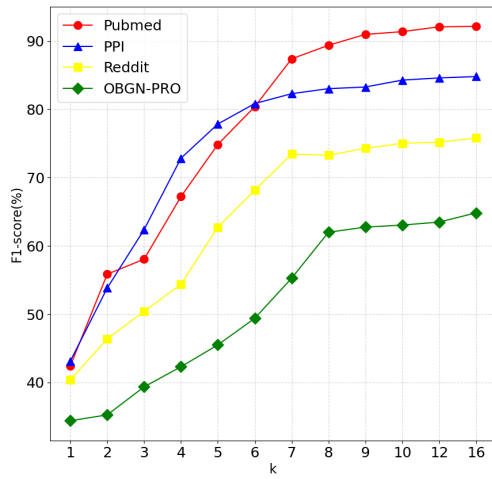
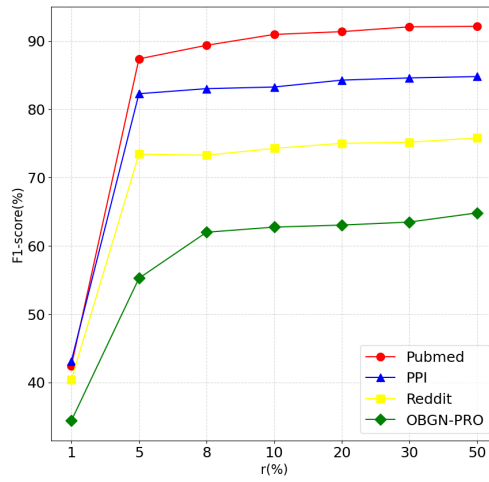
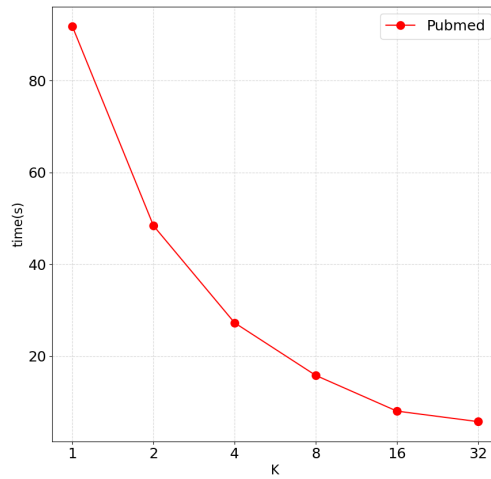
(a) F1 值随 k 变化过程(b) F1 值随 r 变化过程(c) 训练时间随 K 变化过程

图 4-8: 超参分析

4.5 本章小结

在本章节中，我们提出了一种基于图摘要小批量训练的分布式图神经网络训练框架。我们从图神经网络模型的”节点递归地聚合其邻域节点的信息“这一原理出发，通过图摘要技术中的局部敏感哈希算法将大规模图”压缩“为一张张小图，并基于这些子图进行小批量样本的训练模型，从而达到在分布式场景中加速 GNN 模型训练的效果。我们基于三个当下最热门的单机 GNN 模型，借助我们设计的框架实现其分布式训练版本，并在四个真实世界的数据集上对效果进行验证。实验表明，我们提出的分布式图神经网络训练框架能够大幅度加速模型的训练，同时有效节省计算资源。

第五章 总结与展望

5.1 工作总结

本文着力于目前图表征学习领域存在的两大问题——“大规模图数据训练耗时耗资源”与“传统方法难以训练动态图”。本文首先概述了图表示学习领域的相关背景知识，随后对待研究的问题进行了介绍，并对目前针对大规模动态图表示学习的研究工作和算法进行了整理。基于对当前研究工作中的优势与缺陷的分析，我们设计了两种算法——“基于图摘要的大规模时序图表示学习算法”与“基于图摘要的分布式图神经网络训练优化方法”。通过在多个真实世界的大规模动态图数据集上进行了大量实验，我们验证了算法在加速模型训练和减少计算资源损耗上的有效性。

对于“大规模图数据难以训练”的问题，本文首先调研了相关的解决方法，发现现有的解决方法主要围绕两个方面展开：第一，使用采样技术从大规模图中采集出一个规模远小于原图的子图并在其上进行模型训练，达到加速的目的；第二，使用分布式技术，通过增加计算节点的数量来加快训练速度。然而现有方法仍存在一些缺陷。对于采样方法，采样出的子图可能会丢失大量原图的拓扑信息从而导致模型性能的降低，以及采样生成子图的过程是否可以在线性时间内完成。对于分布式方法，分布在不同机器上的邻居节点想要获取对方信息需要发起请求，过多的通信开销会给系统带来极大的负担从而拖慢训练的速度。因此我们分别提出了基于图摘要的大规模时序图表征学习算法和分布式图神经网络训练加速算法，通过图摘要技术中的局部敏感哈希对原图中的节点进行映射从而得到一组规模远小于原图的图摘要。并通过在这些图摘要中使用时序学习方法与分布式技术达到加速模型训练和减少计算资源消耗的目的。通过在多个真实世界中的大规模图数据集上进行的实验显示了该方法的有效性。

对于“传统图表示学习方法难以学习动态图”的问题，本文首先调研了动态图表示学习的相关工作，发现现有的方法大多使用图神经网络与循环神经网络相结合的方式学习动态图的表示，随后我们总结了这类方法的通用框架。

然而在实际应用中，由于循环神经网络的结构特殊性，其要求图中节点不能出现增减的变化，这就对动态图中节点增减这一情景做出了限制。因此我们提出了基于图摘要的动态图表示学习方法，其将原图节点通过一个局部敏感哈希映射到一个规模固定的新图中并学习新图的表示，最后将新图的表示作为循环神经网络的输入进行时序特征的学习，解决了现有方法无法处理动态图中节点变化的问题。大量基于真实世界大规模图数据集上的实验表明，本算法可以有效处理动态图中的时序信息。

5.2 前景展望

近年来图表示学习在现实中的多个领域都已取得了引人注目的成绩，包括金融、生物医学、社交网络，成为了解决链路预测、节点分类问题的有效方法。然而，随着所研究图的规模日益增大，面对动辄数百万节点与上千万条边组成的大规模图，传统的图表示学习方法往往不能有效对其进行训练，而引入采样等技术对原图进行“压缩”后训练的方法又会不可避免地丢失原图拓扑信息，从而使模型的精度下降。如何找到“效率”与“精度”之间的平衡点仍然是一个值得研究的问题。

尽管针对图表示学习已经提出了许多有效的方法，但如何提高所学习到表征的可解释性，即“如何判断训练得到的表征中每一维度特征所包含的具体信息”仍然是一项具有挑战性的任务。

此外，本文的研究只针对图中节点级的表示学习，而没有对边级别的表征乃至图级别的表征进行探索。在未来的研究中，将表示学习的范围扩大到针对边乃至子图可能是一个可行的研究方向。

参考文献

- [1] ABDI H, WILLIAMS L J. Principal component analysis[J]. Wiley interdisciplinary reviews: computational statistics, 2010, 2(4): 433 – 459.
- [2] SAMKO O, MARSHALL A D, ROSIN P L. Selection of the optimal parameter value for the Isomap algorithm[J]. Pattern Recognition Letters, 2006, 27(9): 968 – 979.
- [3] PEROZZI B, AL-RFOU R, SKIENA S. Deepwalk: Online learning of social representations[C] // Proceedings of the 20th ACM SIGKDD international conference on Knowledge discovery and data mining. 2014: 701 – 710.
- [4] BELKIN M, NIYOGI P. Laplacian eigenmaps for dimensionality reduction and data representation[J]. Neural computation, 2003, 15(6): 1373 – 1396.
- [5] CHEN F, WANG Y-C, WANG B, et al. Graph representation learning: a survey[J]. APSIPA Transactions on Signal and Information Processing, 2020, 9: e15.
- [6] HAMILTON W, YING Z, LESKOVEC J. Inductive Representation Learning on Large Graphs[J/OL], 2017, 30.
<https://proceedings.neurips.cc/paper/2017/file/5dd9db5e033da9c6fb5ba83c7a7e9-Paper.pdf>.
- [7] ZENG H, ZHOU H, SRIVASTAVA A, et al. Graphsaint: Graph sampling based inductive learning method[J]. arXiv preprint arXiv:1907.04931, 2019.
- [8] CHEN J, ZHU J, SONG L. Stochastic training of graph convolutional networks with variance reduction[J]. arXiv preprint arXiv:1710.10568, 2017.
- [9] CHIANG W-L, LIU X, SI S, et al. Cluster-gcn: An efficient algorithm for training deep and large graph convolutional networks[C] // Proceedings of the 25th ACM SIGKDD international conference on knowledge discovery & data mining. 2019: 257 – 266.

- [10] KIPF T N, WELLING M. Semi-Supervised Classification with Graph Convolutional Networks[J], 2017.
- [11] DEFFERRARD M, BRESSON X, VANDERGHEYNST P. Convolutional neural networks on graphs with fast localized spectral filtering[J]. Advances in neural information processing systems, 2016, 29.
- [12] DERR T, MA Y, TANG J. Signed graph convolutional networks[C] // 2018 IEEE International Conference on Data Mining (ICDM). 2018 : 929 – 934.
- [13] KINGMA D P, WELLING M. Auto-encoding variational bayes[J]. arXiv preprint arXiv:1312.6114, 2013.
- [14] LEE J B, ROSSI R A, KIM S, et al. Attention models in graphs: A survey[J]. ACM Transactions on Knowledge Discovery from Data (TKDD), 2019, 13(6) : 1 – 25.
- [15] VELIČKOVIĆ P, CUCURULL G, CASANOVA A, et al. Graph attention networks[J]. arXiv preprint arXiv:1710.10903, 2017.
- [16] ABU-EL-HAJIA S, PEROZZI B, AL-RFOU R, et al. Watch your step: Learning node embeddings via graph attention[J]. Advances in neural information processing systems, 2018, 31.
- [17] XU Q, WANG Q, XU C, et al. Attentive graph-based recursive neural network for collective vertex classification[C] // Proceedings of the 2017 ACM on Conference on Information and Knowledge Management. 2017 : 2403 – 2406.
- [18] CHEN F, WANG B, KUO C-C J. Graph-based deep-tree recursive neural network (DTRNN) for text classification[C] // 2018 IEEE Spoken Language Technology Workshop (SLT). 2018 : 743 – 749.
- [19] HAMILTON W, YING Z, LESKOVEC J. Inductive representation learning on large graphs[J]. Advances in neural information processing systems, 2017, 30.
- [20] WANG H, WANG J, WANG J, et al. Graphgan: Graph representation learning with generative adversarial nets[C] // Proceedings of the AAAI conference on artificial intelligence : Vol 32. 2018.

-
- [21] TRIVEDI R, FARAJTABAR M, BISWAL P, et al. Dyrep: Learning representations over dynamic graphs[C] // International conference on learning representations. 2019.
- [22] TRIVEDI R, DAI H, WANG Y, et al. Know-evolve: Deep temporal reasoning for dynamic knowledge graphs[C] // international conference on machine learning. 2017 : 3462 – 3471.
- [23] ZHOU L, YANG Y, REN X, et al. Dynamic network embedding by modeling triadic closure process[C] // Proceedings of the AAAI conference on artificial intelligence : Vol 32. 2018.
- [24] ZUO Y, LIU G, LIN H, et al. Embedding temporal network via neighborhood formation[C] // Proceedings of the 24th ACM SIGKDD international conference on knowledge discovery & data mining. 2018 : 2857 – 2866.
- [25] ROSSI E, CHAMBERLAIN B, FRASCA F, et al. Temporal graph networks for deep learning on dynamic graphs[J]. arXiv preprint arXiv:2006.10637, 2020.
- [26] KUMAR S, ZHANG X, LESKOVEC J. Predicting dynamic embedding trajectory in temporal interaction networks[C] // Proceedings of the 25th ACM SIGKDD international conference on knowledge discovery & data mining. 2019 : 1269 – 1278.
- [27] PAREJA A, DOMENICONI G, CHEN J, et al. Evolvegn: Evolving graph convolutional networks for dynamic graphs[C] // Proceedings of the AAAI conference on artificial intelligence : Vol 34. 2020 : 5363 – 5370.
- [28] MA L, YANG Z, MIAO Y, et al. NeuGraph: Parallel Deep Neural Network Computation on Large Graphs.[C] // USENIX Annual Technical Conference. 2019 : 443 – 458.
- [29] JIA Z, LIN S, GAO M, et al. Improving the accuracy, scalability, and performance of graph neural networks with roc[J]. Proceedings of Machine Learning and Systems, 2020, 2 : 187 – 198.

- [30] CAI Z, YAN X, WU Y, et al. DGCL: an efficient communication library for distributed GNN training[C] // Proceedings of the Sixteenth European Conference on Computer Systems. 2021 : 130 – 144.
- [31] WANG L, YIN Q, TIAN C, et al. FlexGraph: a flexible and efficient distributed framework for GNN training[C] // Proceedings of the Sixteenth European Conference on Computer Systems. 2021 : 67 – 82.
- [32] ZHANG D, HUANG X, LIU Z, et al. Agl: a scalable system for industrial-purpose graph machine learning[J]. arXiv preprint arXiv:2003.02454, 2020.
- [33] KALER T, STATHAS N, OUYANG A, et al. Accelerating training and inference of graph neural networks with fast sampling and pipelining[J]. Proceedings of Machine Learning and Systems, 2022, 4 : 172 – 189.
- [34] ZHU R, ZHAO K, YANG H, et al. Aligraph: A comprehensive graph neural network platform[J]. arXiv preprint arXiv:1902.08730, 2019.
- [35] ZHANG L, LAI Z, LI S, et al. 2pgraph: Accelerating gnn training over large graphs on gpu clusters[C] // 2021 IEEE International Conference on Cluster Computing (CLUSTER). 2021 : 103 – 113.
- [36] LIU Y, SAFAVI T, DIGHE A, et al. Graph summarization methods and applications: A survey[J]. ACM computing surveys (CSUR), 2018, 51(3) : 1 – 34.
- [37] NAVLAKHA S, RASTOGI R, SHRIVASTAVA N. Graph summarization with bounded error[C] // Proceedings of the 2008 ACM SIGMOD international conference on Management of data. 2008 : 419 – 432.
- [38] SHEN Z, MA K-L, ELIASSI-RAD T. Visual analysis of large heterogeneous social networks by semantic and structural abstraction[J]. IEEE transactions on visualization and computer graphics, 2006, 12(6) : 1427 – 1439.
- [39] TANG N, CHEN Q, MITRA P. Graph stream summarization: From big bang to big crunch[C] // Proceedings of the 2016 International Conference on Management of Data. 2016 : 1481 – 1496.

-
- [40] LI J, DANI H, HU X, et al. Attributed network embedding for learning in a dynamic environment[C] // Proceedings of the 2017 ACM on Conference on Information and Knowledge Management. 2017 : 387–396.
- [41] SEO Y, DEFFERRARD M, VANDERGHEYNST P, et al. Structured sequence modeling with graph convolutional recurrent networks[C] // Neural Information Processing: 25th International Conference, ICONIP 2018, Siem Reap, Cambodia, December 13-16, 2018, Proceedings, Part I 25. 2018 : 362–373.
- [42] ZHANG C, JAMES J, LIU Y. Spatial-temporal graph attention networks: A deep learning approach for traffic forecasting[J]. IEEE Access, 2019, 7 : 166246–166256.
- [43] CHEN J, MA T, XIAO C. Fastgcn: fast learning with graph convolutional networks via importance sampling[J]. arXiv preprint arXiv:1801.10247, 2018.
- [44] YING R, HE R, CHEN K, et al. Graph convolutional neural networks for web-scale recommender systems[C] // Proceedings of the 24th ACM SIGKDD international conference on knowledge discovery & data mining. 2018 : 974–983.
- [45] LESKOVEC J, FALOUTSOS C. Sampling from large graphs[C] // Proceedings of the 12th ACM SIGKDD international conference on Knowledge discovery and data mining. 2006 : 631–636.
- [46] PAGE L, BRIN S, MOTWANI R, et al. The PageRank citation ranking: Bringing order to the web.[R]. [S.l.] : Stanford InfoLab, 1999.
- [47] RIBEIRO B, TOWSLEY D. Estimating and sampling graphs with multidimensional random walks[C] // Proceedings of the 10th ACM SIGCOMM conference on Internet measurement. 2010 : 390–403.
- [48] JIN L, CHEN Y, HUI P, et al. Albatross sampling: robust and effective hybrid vertex sampling for social graphs[C] // Proceedings of the 3rd ACM international workshop on MobiArch. 2011 : 11–16.
- [49] CHARIKAR M S. Similarity estimation techniques from rounding algorithms[C] // Proceedings of the thirty-fourth annual ACM symposium on Theory of computing. 2002 : 380–388.

- [50] WANG C, CHEN L. Continuous subgraph pattern search over graph streams[C] // 2009 IEEE 25th International Conference on Data Engineering. 2009 : 393 – 404.
- [51] GRAVES A, GRAVES A. Long short-term memory[J]. Supervised sequence labelling with recurrent neural networks, 2012 : 37 – 45.
- [52] DEY R, SALEM F M. Gate-variants of gated recurrent unit (GRU) neural networks[C] // 2017 IEEE 60th international midwest symposium on circuits and systems (MWSCAS). 2017 : 1597 – 1600.
- [53] WANG W, YANG N, WEI F, et al. Gated self-matching networks for reading comprehension and question answering[C] // Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers). 2017 : 189 – 198.
- [54] NAMATA G, LONDON B, GETOOR L, et al. Query-driven active surveying for collective classification[C] // 10th international workshop on mining and learning with graphs : Vol 8. 2012 : 1.
- [55] LU Y, WANG X, SHI C, et al. Temporal network embedding with micro-and macro-dynamics[C] // Proceedings of the 28th ACM international conference on information and knowledge management. 2019 : 469 – 478.
- [56] GROVER A, LESKOVEC J. node2vec: Scalable feature learning for networks[C] // Proceedings of the 22nd ACM SIGKDD international conference on Knowledge discovery and data mining. 2016 : 855 – 864.
- [57] PANAGOPOULOS G, NIKOLENTZOS G, VAZIRGIANNIS M. Transfer graph neural networks for pandemic forecasting[C] // Proceedings of the AAAI Conference on Artificial Intelligence : Vol 35. 2021 : 4838 – 4845.
- [58] XU D, RUAN C, KORPEOGLU E, et al. Inductive representation learning on temporal graphs[J]. arXiv preprint arXiv:2002.07962, 2020.
- [59] Van der MAATEN L, HINTON G. Visualizing data using t-SNE.[J]. Journal of machine learning research, 2008, 9(11).

- [60] SONG Z, BAI F, ZHAO J, et al. Spammer detection using graph-level classification model of graph neural network[C] // 2021 IEEE 2nd International Conference on Big Data, Artificial Intelligence and Internet of Things Engineering (ICBAIE). 2021 : 531 – 538.
- [61] TAN Q, LIU N, HU X. Deep representation learning for social network analysis[J]. *Frontiers in big Data*, 2019, 2 : 2.
- [62] FOUT A, BYRD J, SHARIAT B, et al. Protein interface prediction using graph convolutional networks[J]. *Advances in neural information processing systems*, 2017, 30.
- [63] FAN W, MA Y, LI Q, et al. Graph neural networks for social recommendation[C] // *The world wide web conference*. 2019 : 417 – 426.
- [64] HU L, XU S, LI C, et al. Graph neural news recommendation with unsupervised preference disentanglement[C] // *Proceedings of the 58th annual meeting of the association for computational linguistics*. 2020 : 4255 – 4264.
- [65] HUANG T, DONG Y, DING M, et al. Mixgcf: An improved training method for graph neural network-based recommender systems[C] // *Proceedings of the 27th ACM SIGKDD Conference on Knowledge Discovery & Data Mining*. 2021 : 665 – 674.
- [66] WU S, SUN F, ZHANG W, et al. Graph neural networks in recommender systems: a survey[J]. *ACM Computing Surveys*, 2022, 55(5) : 1 – 37.
- [67] ZHANG Y, YU X, CUI Z, et al. Every document owns its structure: Inductive text classification via graph neural networks[J]. *arXiv preprint arXiv:2004.13826*, 2020.
- [68] ZHONG W, XU J, TANG D, et al. Reasoning over semantic-level graph for fact checking[J]. *arXiv preprint arXiv:1909.03745*, 2019.
- [69] BATTAGLIA P W, HAMRICK J B, BAPST V, et al. Relational inductive biases, deep learning, and graph networks[J]. *arXiv preprint arXiv:1806.01261*, 2018.

- [70] CEN Y, HOU Z, WANG Y, et al. Cogdl: An extensive toolkit for deep learning on graphs[J]. arXiv preprint arXiv:2103.00959, 2021 : 1 – 11.
- [71] FEY M, LENSSEN J E. Fast graph representation learning with PyTorch Geometric[J]. arXiv preprint arXiv:1903.02428, 2019.
- [72] GRATTAROLA D, ALIPPI C. Graph neural networks in tensorflow and keras with spektral [application notes][J]. IEEE Computational Intelligence Magazine, 2021, 16(1): 99 – 106.
- [73] CHEN Z, YAN M, ZHU M, et al. fuseGNN: Accelerating graph convolutional neural network training on GPGPU[C] // Proceedings of the 39th International Conference on Computer-Aided Design. 2020 : 1 – 9.
- [74] FU Q, JI Y, HUANG H H. TLPGNN: A lightweight two-level parallelism paradigm for graph neural network computation on GPU[C] // Proceedings of the 31st International Symposium on High-Performance Parallel and Distributed Computing. 2022 : 122 – 134.
- [75] HU Y, YE Z, WANG M, et al. Featgraph: A flexible and efficient backend for graph neural network systems[C] // SC20: International Conference for High Performance Computing, Networking, Storage and Analysis. 2020 : 1 – 13.
- [76] WU Y, MA K, CAI Z, et al. Seastar: vertex-centric programming for graph neural networks[C] // Proceedings of the Sixteenth European Conference on Computer Systems. 2021 : 359 – 375.
- [77] HU W, FEY M, ZITNIK M, et al. Open graph benchmark: Datasets for machine learning on graphs[J]. Advances in neural information processing systems, 2020, 33 : 22118 – 22133.
- [78] BOJCHEVSKI A, KLICPERA J, PEROZZI B, et al. Scaling graph neural networks with approximate pagerank[C] // Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining. 2020 : 2464 – 2473.
- [79] LIU X, YAN M, DENG L, et al. Survey on graph neural network acceleration: An algorithmic perspective[J]. arXiv preprint arXiv:2202.04822, 2022.

- [80] YOU Y, CHEN T, WANG Z, et al. L2-gcn: Layer-wise and learned efficient training of graph convolutional networks[C] // Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition. 2020 : 2127 – 2135.
- [81] DENG X, ZHANG Z. Graph-free knowledge distillation for graph neural networks[J]. arXiv preprint arXiv:2105.07519, 2021.
- [82] YAN B, WANG C, GUO G, et al. Tinygcn: Learning efficient graph neural networks[C] // Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining. 2020 : 1848 – 1856.
- [83] ZHANG W, MIAO X, SHAO Y, et al. Reliable data distillation on graph convolutional network[C] // Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data. 2020 : 1399 – 1414.
- [84] LIN H, YAN M, YE X, et al. A Comprehensive Survey on Distributed Training of Graph Neural Networks[J]. arXiv preprint arXiv:2211.05368, 2022.
- [85] MD V, MISRA S, MA G, et al. Distgcn: Scalable distributed training for large-scale graph neural networks[C] // Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis. 2021 : 1 – 14.
- [86] MOSTAFA H. Sequential aggregation and rematerialization: Distributed full-batch training of graph neural networks on large graphs[J]. Proceedings of Machine Learning and Systems, 2022, 4 : 265 – 275.
- [87] LIU Y, LI H, ZHANG G, et al. GraphTheta: A distributed graph neural network learning system with flexible training strategy[J]. arXiv preprint arXiv:2104.10569, 2021.
- [88] STANTON I, KLIOT G. Streaming graph partitioning for large distributed graphs[C] // Proceedings of the 18th ACM SIGKDD international conference on Knowledge discovery and data mining. 2012 : 1222 – 1230.
- [89] XIE C, YAN L, LI W-J, et al. Distributed power-law graph computing: Theoretical and empirical analysis[J]. Advances in neural information processing systems, 2014, 27.

-
- [90] ZHOU J, CUI G, HU S, et al. Graph neural networks: A review of methods and applications[J]. AI open, 2020, 1 : 57–81.
- [91] ZHENG D, MA C, WANG M, et al. Distdgl: distributed graph neural network training for billion-scale graphs[C] // 2020 IEEE/ACM 10th Workshop on Irregular Applications: Architectures and Algorithms (IA3). 2020 : 36–44.

致 谢

简历与科研成果

基本信息

教育背景

攻读硕士学位期间完成的学术成果

1. 南京大学学报 (自然科学), 北大核心期刊, 已接收, 第一作者
2. Thirty-Seventh AAAI Conference on Artificial Intelligence (AAAI 2023). CCF-A 类会议, 已接收, 第四作者
3. The ACM Web Conference Proceedings (WWW 2022). CCF-A 类会议, 已接收, 第四作者
4. Pacific-Asia Conference on Knowledge Discovery and Data Mining (PAKDD 2021). CCF-C 类会议, 已接收, 第三作者
5. 14th Asian Conference on Machine Learning (ACML 2022). CCF-C 类会议, 已接收, 第四作者

攻读硕士学位期间参与的科研课题

1. 基于数据流分析的电力互联网应用运维技术研究 5700-202153172A-0-0-00 国家电网科技项目 2021.1-2022.12
2. 基于深度学习的异构无线网络资源调度优化研究 61972196 国家自然科学基金面上项目 2020.1-2023.12

版权及论文原创性说明

任何收存和保管本论文的单位和个人，未经作者本人授权，不得将本论文转借他人并复印、抄录、拍照或以任何方式传播，否则，引起有碍作者著作权益的问题，将可能承担法律责任。

本人郑重声明：所呈交的学位论文，是本人在导师的指导下，独立进行研究工作所取得的成果。除文中已经注明引用的内容外，本论文不包含其他个人或集体已经发表或撰写的作品成果。本文所引用的重要文献，均已在文中以明确方式标明。本声明的法律结果由本人承担。

作者签名：_____
_____年____月____日

《学位论文出版授权书》

本人完全同意《中国优秀博硕士学位论文全文数据库出版章程》（以下简称“章程”），愿意将本人的学位论文提交“中国学术期刊（光盘版）电子杂志社”在《中国博士学位论文全文数据库》、《中国优秀硕士学位论文全文数据库》中全文发表。《中国博士学位论文全文数据库》、《中国优秀硕士学位论文全文数据库》可以以电子、网络及其他数字媒体形式公开出版，并同意编入《中国知识资源总库》，在《中国博硕士学位论文评价数据库》中使用和在互联网上传播，同意按“章程”规定享受相关权益。

作者签名：_____

_____年____月____日

论文题名	面向大规模动态图的图神经网络优化机制研究				
研究生学号		所在院系	计算机科学与技术系	学位年度	2023
论文级别	☐ 学术学位硕士 ☐ 专业学位硕士 ☐ 学术学位博士 ☐ 专业学位博士 (请在方框内画钩)				
作者 Email					
导师姓名					

论文涉密情况：

☐ 不保密

☐ 保密，保密期(_____年____月____日至 _____年____月____日)

注：请将该授权书填写后装订在学位论文最后一页（南大封面）。

